Combinational Logic Circuits - Adders & Subtractors

1. The Half Adder

Theory: A half adder is a combinational circuit that adds two single-bit binary numbers (A and B). It has two outputs: the **Sum (S)** and the **Carry (C)**. It is called a "half" adder because it cannot accept a carry-in from a previous addition.

Truth Table:

Inputs		Outputs	
A	B	S (Sum)	C (Carry)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

K-Map Simplification for Half Adder

For Sum (S):

Notice the checkerboard pattern. The `1`s cannot be grouped, which is the classic signature of an XOR gate.

$A \backslash B$	0	1
0	0	**1**
1	**1**	0

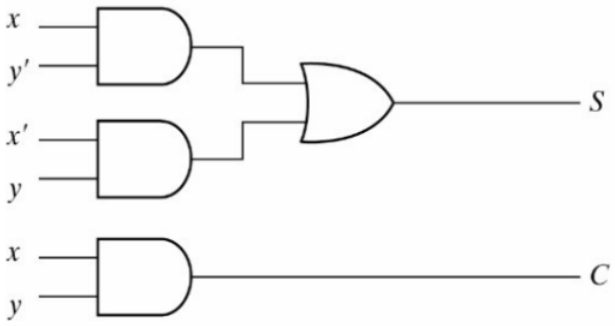
- **Sum Equation:** $S = \bar{A}B + A\bar{B} = \mathbf{A \oplus B}$

For Carry (C):

$A \backslash B$	0	1
0	0	0
1	0	**1**

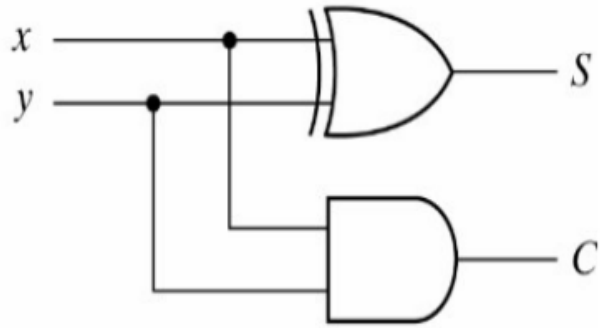
- **Carry Equation:** $C = \mathbf{AB}$

Circuit Diagram for Half Adder



S=XY'+X'Y
C=XY

OR



S = X ⊕ Y
C=XY

2. The Full Adder

Theory:

A full adder improves upon the half adder by adding three single-bit numbers: A, B, and a **Carry-In (C_{in})** from a previous less-significant bit stage. This is the building block needed to add multi-bit binary numbers.

Truth Table:

Inputs			Outputs	
A	B	C_{in}	S (Sum)	C_{out} (Carry Out)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-Map Simplification for Full Adder

For Sum (S):

Again, a perfect 3-variable checkerboard. No groupings are possible.

$A \backslash BC_{in}$	00	01	11	10
0	0	**1**	0	**1**
1	**1**	0	**1**	0

- **Sum Equation:** $S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in} = \mathbf{A \oplus B \oplus C_{in}}$

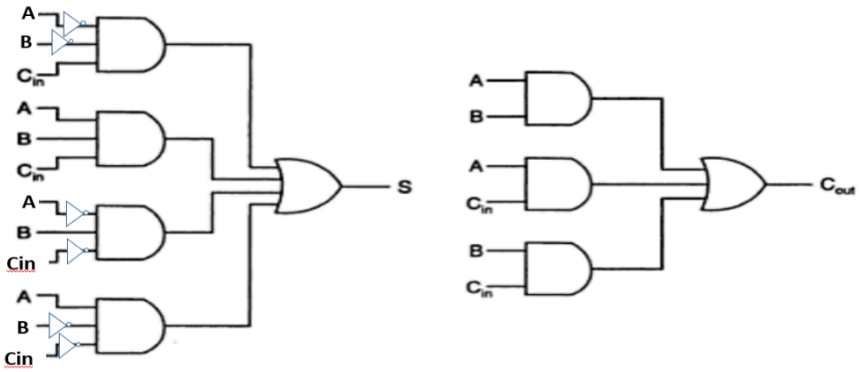
For Carry-Out (C_{out}):

Here we can form three separate groups of two.

$A \backslash BC_{in}$	00	01	11	10
0	0	0	**1**	0
1	0	**1**	**1**	**1**

- **Carry-Out Equation:** $C_{out} = \mathbf{AB + BC_{in} + AC_{in}}$

Circuit Diagram for Full Adder



$S = A'B'C_{in} + ABC_{in} + AB'C_{in} + A'BC_{in}$

$C = BC_{in} + AB_{in} + AC_{in}$

---

3. The Half Subtractor

Theory:

A half subtractor performs $A - B$. It produces a **Difference (D)** and a **Borrow (B_{out})**. The borrow is triggered (set to 1) only when you try to subtract 1 from 0.

Truth Table:

Inputs		Outputs	
A	B	D (Difference)	B_{out} (Borrow)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

K-Map Simplification for Half Subtractor

For Difference (D):

Identical to the Half Adder Sum!

$A \backslash B$	0	1
0	0	**1**
1	**1**	0

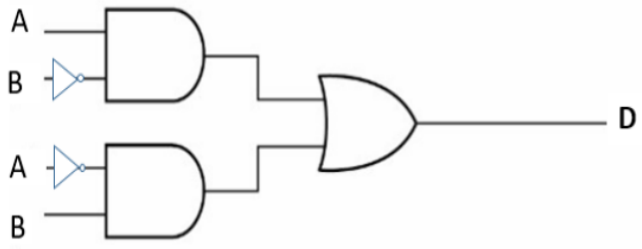
- **Difference Equation:** $D = \bar{A}B + A\bar{B} = \mathbf{A \oplus B}$

For Borrow (B_{out}):

$A \backslash B$	0	1
0	0	**1**
1	0	0

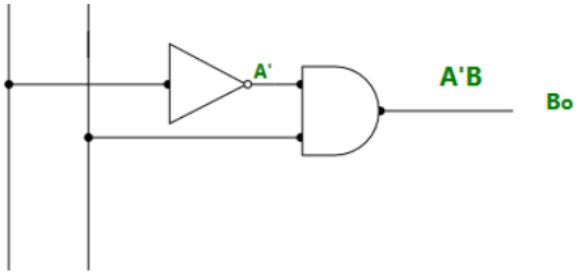
- **Borrow Equation:** $B_{out} = \mathbf{\bar{A}B}$

Circuit Diagram for Half Subtractor



OR

A B



---

4. The Full Subtractor

Theory:

A full subtractor performs $(A - B - B_{in})$, taking into account a **Borrow-In (B_{in})** requested by a previous, less-significant stage.

Truth Table:

Inputs			Outputs	
A	B	B_{in}	D (Difference)	B_{out} (Borrow Out)
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-Map Simplification for Full Subtractor

For Difference (D):

Identical to the Full Adder Sum!

$A\backslash BB_{in}$	00	01	11	10
0	0	**1**	0	**1**
1	**1**	0	**1**	0

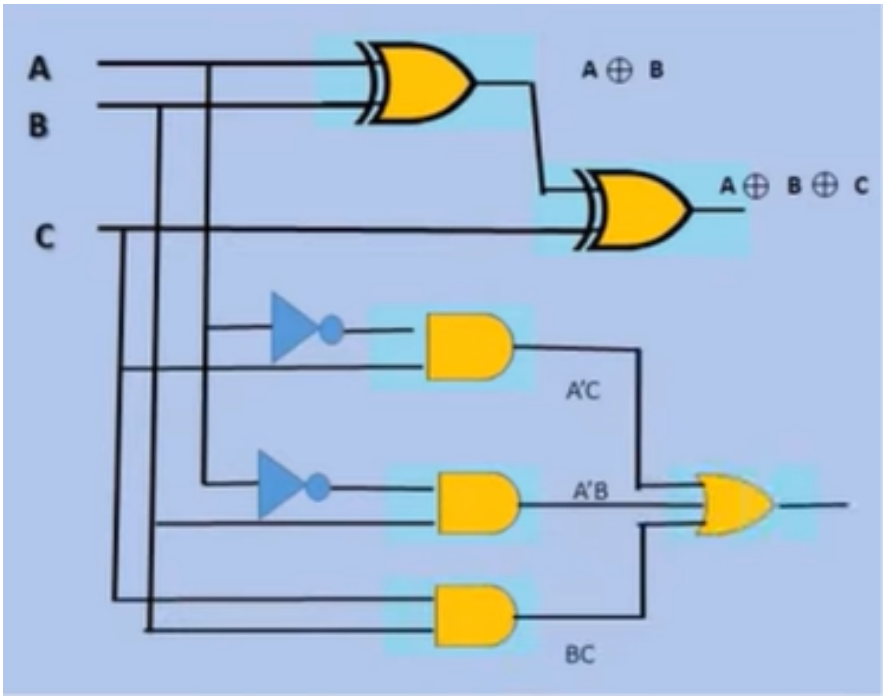
- **Difference Equation:** $D = \mathbf{A} \oplus \mathbf{B} \oplus \mathbf{B_{in}}$

For Borrow-Out (B_{out}):

$A\backslash BB_{in}$	00	01	11	10
0	0	**1**	**1**	**1**
1	0	0	**1**	0

- **Borrow-Out Equation:** $B_{out} = \mathbf{\bar{A}B} + \mathbf{\bar{A}B_{in}} + \mathbf{BB_{in}}$

Circuit Diagram for Full Subtractor



Combinational Logic - Multiplexers (MUX)

1. Introduction to Multiplexers

- A digital logic circuit that is capable of accepting several inputs and generating a single output is known as a multiplexer or MUX.
- The multiplexer functions as a data selector that takes many inputs and provides one selected output.
- In a multiplexer, there are 2^n input lines and 1 output line, where n represents the number of select lines.
- Because of this specific architecture, the multiplexer is commonly referred to as a "many-to-one" circuit.

Types of Multiplexers: Multiplexers are primarily classified into four types based on their input and select line configurations:

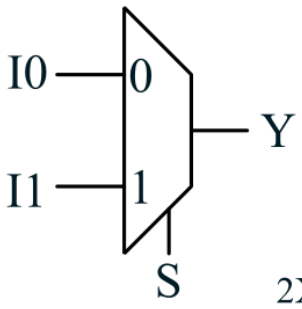
1. 2-to-1 multiplexer (1 select line)
2. 4-to-1 multiplexer (2 select lines)
3. 8-to-1 multiplexer (3 select lines)
4. 16-to-1 multiplexer (4 select lines)

2. The 2x1 Multiplexer

- The 2×1 MUX is a fundamental circuit used to choose one signal from two inputs and transmit it to the output.
- It consists of two input lines (I_0, I_1), one output line (Y), and a single selection line (S).

Truth Table:

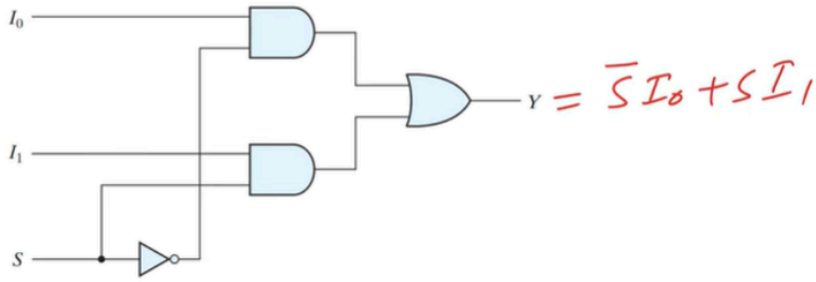
S (Select)	Y (Output)
0	I_0
1	I_1



2X1 MUX

Boolean Expression: When the select line S is logic 0, I_0 is selected, and when S is logic 1, I_1 is selected.

$$Y = \overline{S}I_0 + SI_1$$



3. The 4x1 Multiplexer

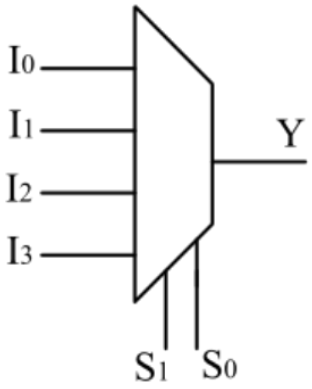
- A 4×1 MUX has four data inputs (I_0, I_1, I_2, I_3) and only one output (Y).
- It requires 2 select lines (S_1, S_0) for operation.

Logic Rules:

- When both select lines are logic 0 ($S_1 = 0, S_0 = 0$), I_0 is selected.
- When S_1 is logic 0 and S_0 is logic 1, I_1 is selected.
- When S_1 is logic 1 and S_0 is logic 0, I_2 is selected.
- When both select lines are logic 1 ($S_1 = 1, S_0 = 1$), I_3 is selected.

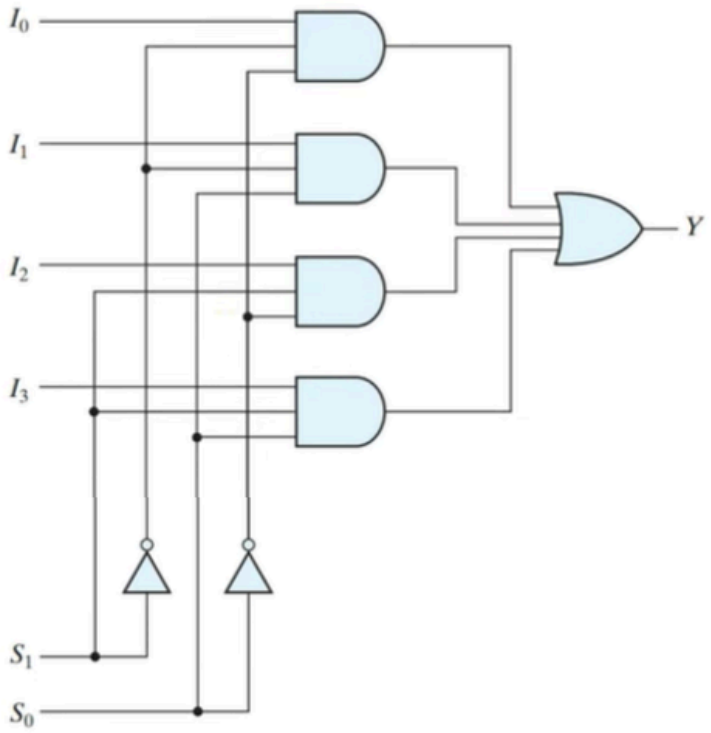
Truth Table:

S_1	S_0	Y (Output)
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



Boolean Expression:

$$Y = \overline{S}_1\overline{S}_0I_0 + \overline{S}_1S_0I_1 + S_1\overline{S}_0I_2 + S_1S_0I_3$$

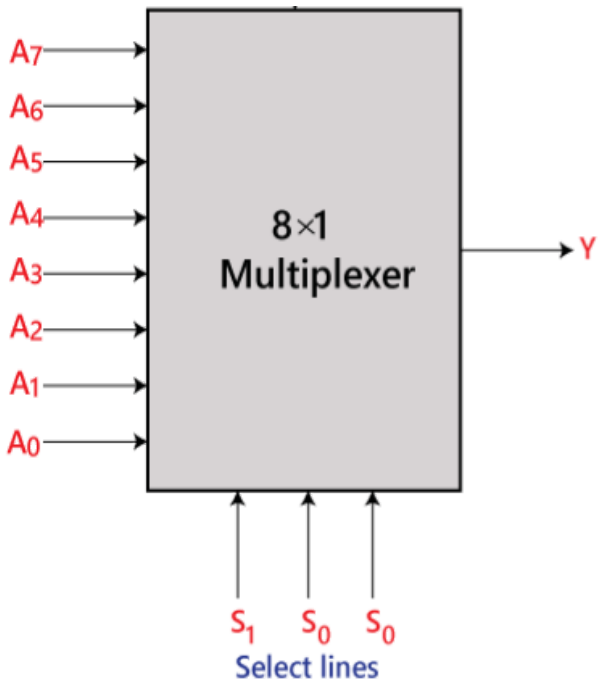

I_0
I_1
I_2
I_3
S_1
S_0
Y

4. The 8x1 Multiplexer

- In the 8×1 multiplexer, there are eight inputs (A_0 to A_7), 3 selection lines (S_2, S_1, S_0), and a single output (Y).
- Based on the binary combination present at the selection lines, one of the 8 inputs is connected to the output.

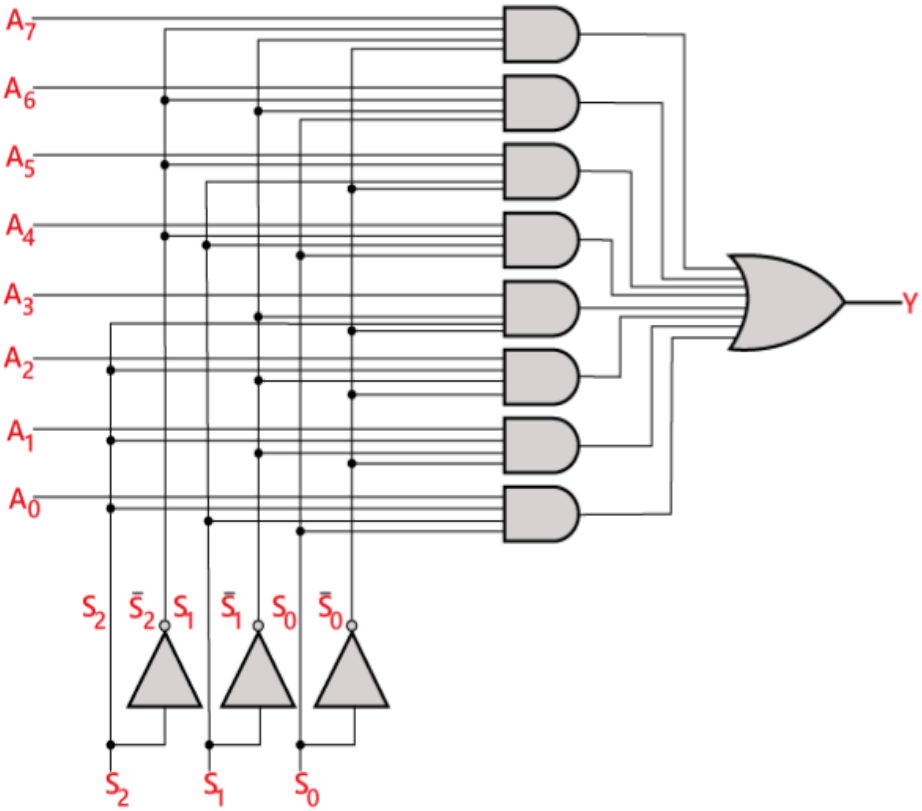
Truth Table:

S_2	S_1	S_0	Y (Output)
0	0	0	A_0
0	0	1	A_1
0	1	0	A_2
0	1	1	A_3
1	0	0	A_4
1	0	1	A_5
1	1	0	A_6
1	1	1	A_7



Boolean Expression:

$$Y = \overline{S}_2\overline{S}_1\overline{S}_0A_0 + \overline{S}_2\overline{S}_1S_0A_1 + \overline{S}_2S_1\overline{S}_0A_2 + \overline{S}_2S_1S_0A_3 + S_2\overline{S}_1\overline{S}_0A_4 + S_2\overline{S}_1S_0A_5 + S_2S_1\overline{S}_0A_6 + S_2S$$

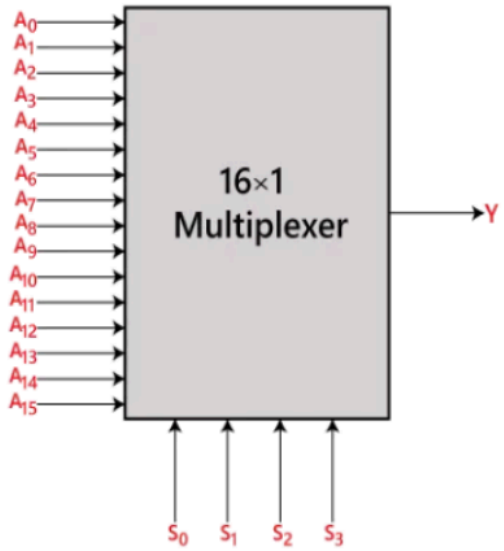

A7
A6
A5
A4
A3
A2
A1
A0
Y
S2
S̄2
S1
S̄1
S0
S̄0
S2
S1
S0

5. The 16x1 Multiplexer

- The 16×1 multiplexer has 16 inputs (A_0 to A_{15}), 4 selection lines (S_3, S_2, S_1, S_0), and a single output (Y).
- Based on the specific combination of inputs present at the selection lines, one of the 16 inputs will be routed to the output.

Truth Table:

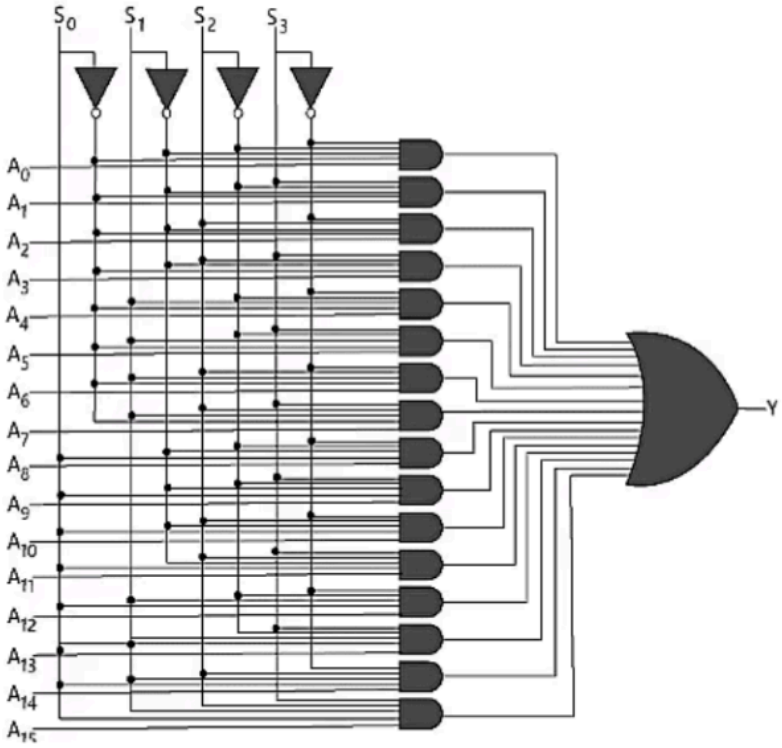
S_3	S_2	S_1	S_0	Y (Output)
0	0	0	0	A_0
0	0	0	1	A_1
0	0	1	0	A_2
0	0	1	1	A_3
0	1	0	0	A_4
0	1	0	1	A_5
0	1	1	0	A_6
0	1	1	1	A_7
1	0	0	0	A_8
1	0	0	1	A_9
1	0	1	0	A_{10}
1	0	1	1	A_{11}
1	1	0	0	A_{12}
1	1	0	1	A_{13}
1	1	1	0	A_{14}
1	1	1	1	A_{15}



16-1 Multiplexer

Boolean Expression:

$$Y = \overline{S}_3\overline{S}_2\overline{S}_1\overline{S}_0A_0 + \overline{S}_3\overline{S}_2\overline{S}_1S_0A_1 + \overline{S}_3\overline{S}_2S_1\overline{S}_0A_2 + \ldots + S_3S_2S_1S_0A_{15}$$



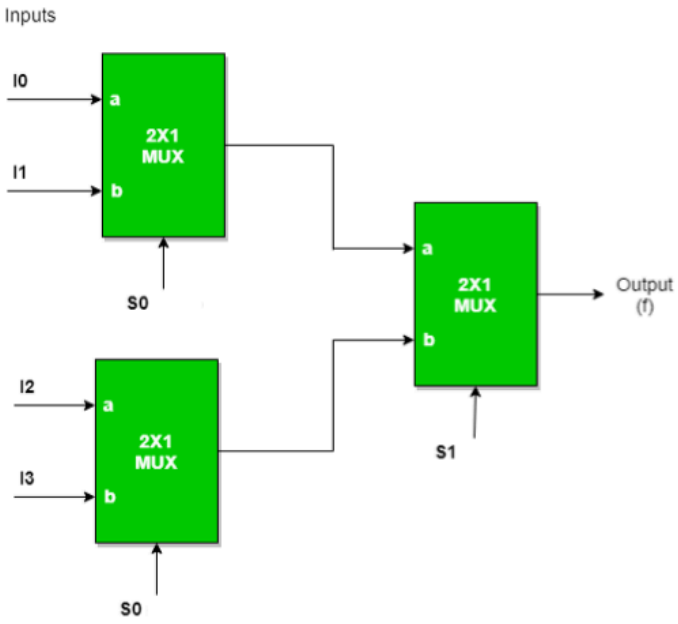
(Expands to cover all 16 minterms associated with their respective data lines).---

6. Multiplexer Implementation (Cascading)

Higher-order multiplexers can be implemented using combinations of lower-order multiplexers. The mathematical approach to finding the number of required MUXes is defined by successive division of data lines.

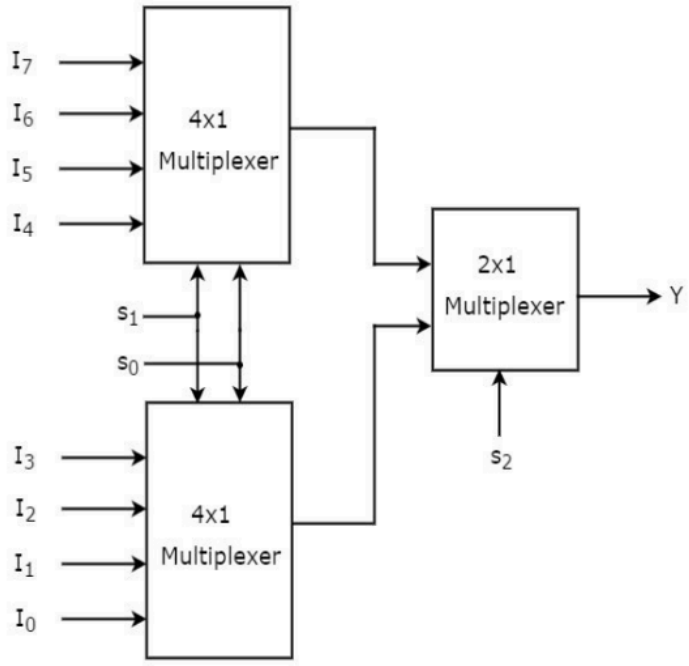
Implementation 1: 4:1 MUX using 2:1 MUX

- **Required MUX:** 4:1 ($n = 4$ data lines)
- **Available MUX:** 2:1 ($p = 2$ data lines)
- **Stage 1 (f):** $n/p = 4/2 = 2$
- **Stage 2 (s):** $f/p = 2/2 = 1$
- **Total 2:1 MUXes Required:** $f + s = 2 + 1 = 3$



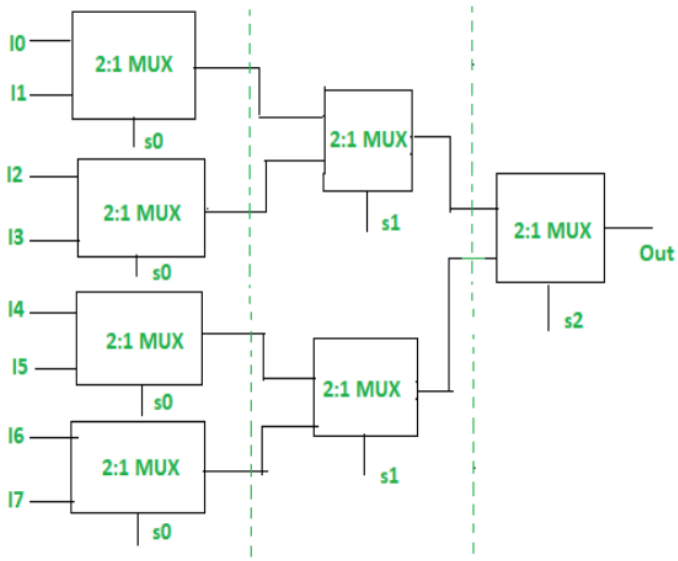
Implementation 2: 8x1 MUX using 4x1 and 2x1 MUX

- We need two 4×1 multiplexers and one 2×1 multiplexer.
- **Stage 1:** For getting 8 data inputs, we need two 4×1 multiplexers (8 / 4 = 2).
- **Stage 2:** The 4×1 multiplexers produce two outputs total. To get the final output, we need one 2×1 multiplexer (2 / 2 = 1).
- **Total Required:** Two 4×1 MUXes and one 2×1 MUX.



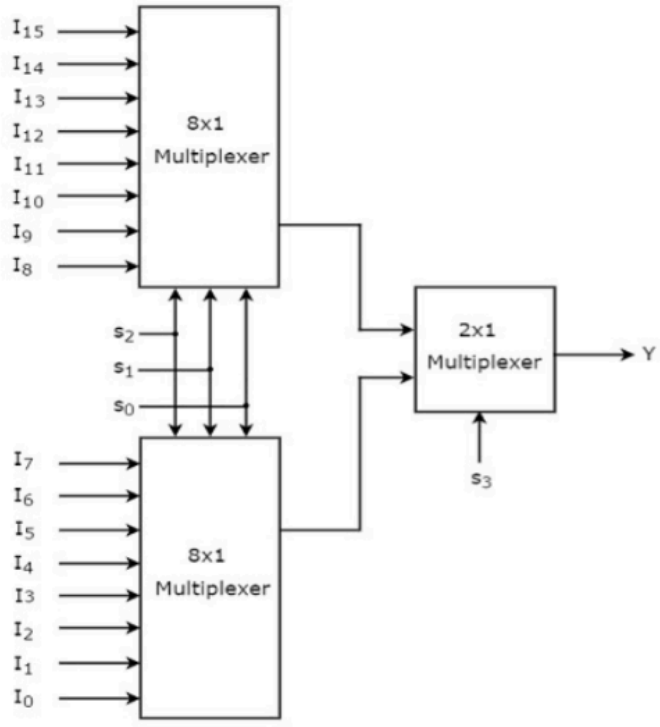
Implementation 3: 8x1 MUX using only 2x1 MUX

- **Required MUX:** 8:1 ($n = 8$ data lines)
- **Available MUX:** 2:1 ($p = 2$ data lines)
- **Stage 1 (f):** $n/p = 8/2 = 4$
- **Stage 2 (s):** $f/p = 4/2 = 2$
- **Stage 3 (t):** $s/p = 2/2 = 1$
- **Total 2:1 MUXes Required:** $f + s + t = 4 + 2 + 1 = 7$



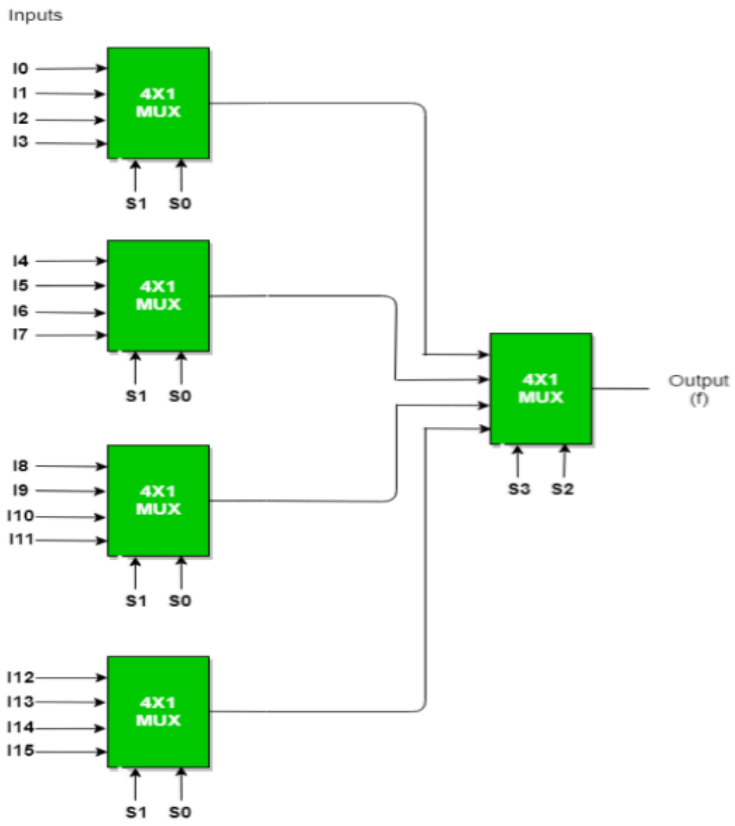
Implementation 4: 16:1 MUX using 8:1 and 2:1 MUX

- **Required MUX:** 16:1 ($n = 16$ data lines)
- **Available MUXes:** 8:1 ($p_1 = 8$) and 2:1 ($p_2 = 2$)
- **Stage 1:** $n/p_1 = 16/8 = 2$ (Requires two 8:1 MUXes)
- **Stage 2:** $f/p_2 = 2/2 = 1$ (Requires one 2:1 MUX)
- **Total Required:** Two 8:1 MUXes and one 2:1 MUX



Implementation 5: 16x1 MUX using only 4x1 MUX

- **Required MUX:** 16:1 ($n = 16$ data lines)
- **Available MUX:** 4:1 ($p = 4$ data lines)
- **Stage 1 (f):** $n/p = 16/4 = 4$
- **Stage 2 (s):** $f/p = 4/4 = 1$
- **Total 4:1 MUXes Required:** $f + s = 4 + 1 = 5$



Combinational Logic - Demultiplexers (DEMUX)

1. Introduction to Demultiplexers

A digital logic circuit that is capable of accepting a single input and distributing it to several outputs is known as a demultiplexer or DEMUX.

- The demultiplexer functions as a data distributor that takes one input and routes it to one selected output line.
- In a demultiplexer, there is 1 input line and 2^n output lines, where n represents the number of select lines.
- Because of this specific architecture, the demultiplexer is commonly referred to as a "one-to-many" circuit.

Types of Demultiplexers: Demultiplexers are primarily classified into four types based on their output and select line configurations:

1. 1-to-2 demultiplexer (1 select line)
2. 1-to-4 demultiplexer (2 select lines)
3. 1-to-8 demultiplexer (3 select lines)
4. 1-to-16 demultiplexer (4 select lines)

2. The 1x2 Demultiplexer

The 1×2 DEMUX is a fundamental circuit used to connect one input signal to one of two output lines.
Untitled

- It consists of one input line (I), two output lines (Y_0, Y_1), and a single selection line (S).

Truth Table:

S (Select)	Y_0	Y_1
0	I	0
1	0	I

Boolean Expression: When the select line S is logic 0, output Y_0 is selected to receive the input I. When S is logic 1, output Y_1 receives the input I.

$$Y_0 = \overline{S}I$$

$$Y_1 = SI$$

3. The 1x4 Demultiplexer

A 1×4 DEMUX has one data input (I) and four outputs (Y_0, Y_1, Y_2, Y_3).

- It requires 2 select lines (S_1, S_0) for operation.

Logic Rules:

- When both select lines are logic 0 ($S_1 = 0, S_0 = 0$), Y_0 receives I.
- When S_1 is logic 0 and S_0 is logic 1, Y_1 receives I.
- When S_1 is logic 1 and S_0 is logic 0, Y_2 receives I.
- When both select lines are logic 1 ($S_1 = 1, S_0 = 1$), Y_3 receives I.

Truth Table:

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

Boolean Expression:

$$Y_0 = \overline{S}_1 \overline{S}_0 I$$

$$Y_1 = \overline{S}_1 S_0 I$$

$$Y_2 = S_1 \overline{S}_0 I$$

$$Y_3 = S_1 S_0 I$$

4. The 1x8 Demultiplexer

In the 1×8 demultiplexer, there is one input (I), 3 selection lines (S_2, S_1, S_0), and eight outputs (Y_0 to Y_7).

- Based on the binary combination present at the selection lines, the input is connected to one specific output among the 8 available lines.

Truth Table:

S_2	S_1	S_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	I	0	0	0	0	0	0	0
0	0	1	0	I	0	0	0	0	0	0
0	1	0	0	0	I	0	0	0	0	0
0	1	1	0	0	0	I	0	0	0	0
1	0	0	0	0	0	0	I	0	0	0
1	0	1	0	0	0	0	0	I	0	0
1	1	0	0	0	0	0	0	0	I	0
1	1	1	0	0	0	0	0	0	0	I

Boolean Expression:

$$Y_0 = \overline{S}_2\overline{S}_1\overline{S}_0 I$$

$$Y_1 = \overline{S}_2\overline{S}_1 S_0 I$$

$$Y_2 = \overline{S}_2 S_1\overline{S}_0 I$$

$$Y_3 = \overline{S}_2 S_1 S_0 I$$

$$Y_4 = S_2\overline{S}_1\overline{S}_0 I$$

$$Y_5 = S_2\overline{S}_1 S_0 I$$

$$Y_6 = S_2 S_1\overline{S}_0 I$$

$$Y_7 = S_2 S_1 S_0 I$$

5. The 1x16 Demultiplexer

The 1×16 demultiplexer has 1 input (I), 4 selection lines (S_3, S_2, S_1, S_0), and 16 outputs (Y_0 to Y_{15}).

- Based on the specific combination of inputs present at the selection lines, the single input will be routed to one of the 16 outputs.

Truth Table:

(Note: Displayed as the active selected line receiving the input I to maintain conciseness)

S_3	S_2	S_1	S_0	Selected Output ($Y_n = I$)
0	0	0	0	Y_0
0	0	0	1	Y_1
0	0	1	0	Y_2
0	0	1	1	Y_3
0	1	0	0	Y_4
0	1	0	1	Y_5
0	1	1	0	Y_6
0	1	1	1	Y_7
1	0	0	0	Y_8
1	0	0	1	Y_9
1	0	1	0	Y_{10}
1	0	1	1	Y_{11}
1	1	0	0	Y_{12}
1	1	0	1	Y_{13}
1	1	1	0	Y_{14}
1	1	1	1	Y_{15}

Boolean Expression:

$$Y_0 = \overline{S}_3\overline{S}_2\overline{S}_1\overline{S}_0 I$$

$$Y_1 = \overline{S}_3\overline{S}_2\overline{S}_1 S_0 I$$

$$Y_2 = \overline{S}_3\overline{S}_2 S_1\overline{S}_0 I$$

$$\dots$$

$$Y_{15} = S_3 S_2 S_1 S_0 I$$

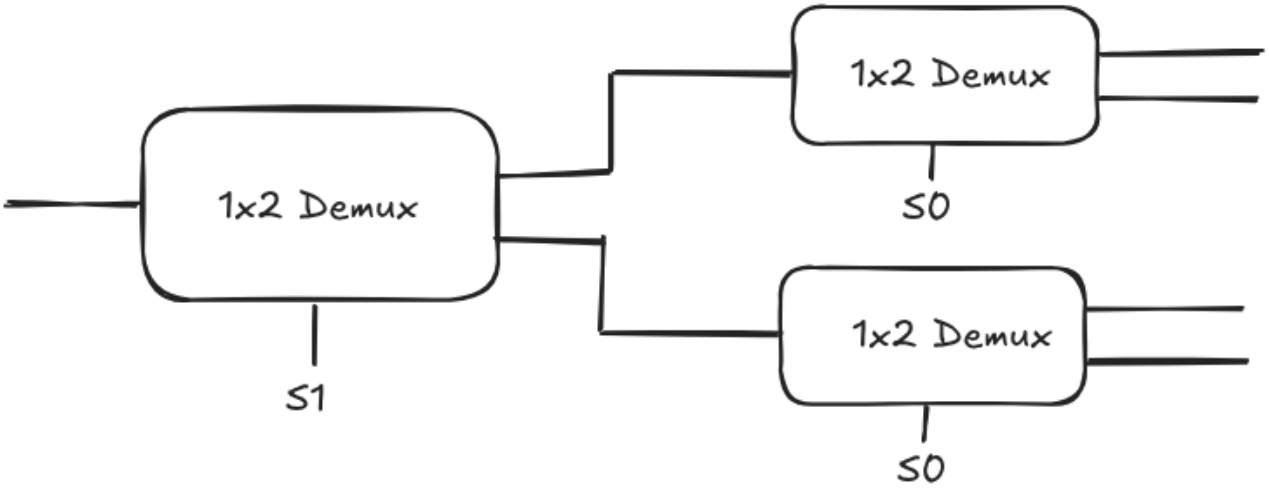
(Expands to cover all 16 minterm equivalent combinations routed with the input I).

6. Demultiplexer Implementation (Cascading)

Higher-order demultiplexers can be implemented using combinations of lower-order demultiplexers. Because DEMUX trees are structurally the exact reverse of MUX trees, the mathematical calculation for the number of required blocks is identical to multiplexers.

Implementation 1: 1:4 DEMUX using 1:2 DEMUX

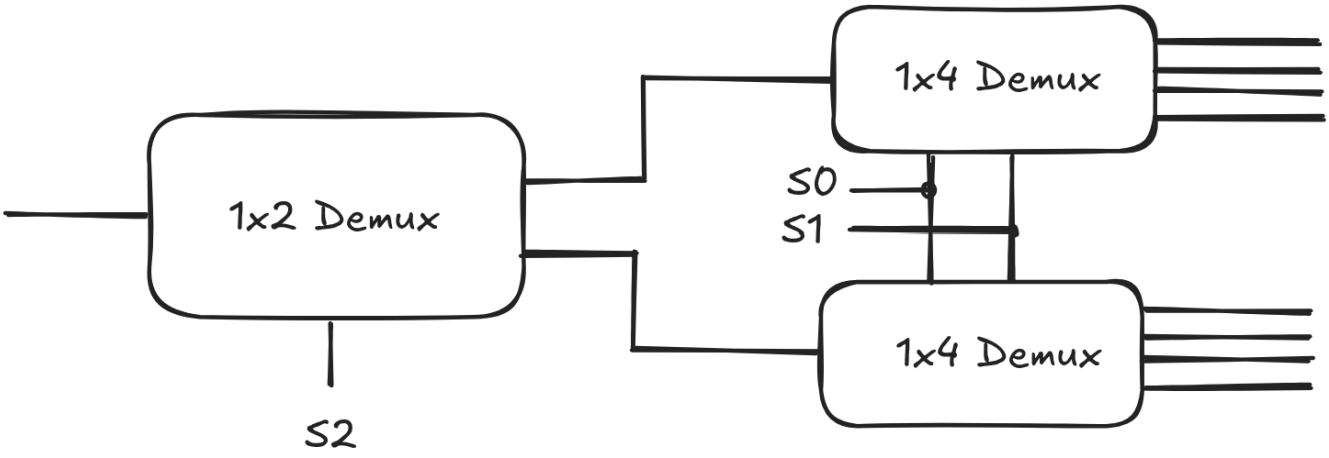
- **Required DEMUX:** 1:4 ($n = 4$ output lines)
- **Available DEMUX:** 1:2 ($p = 2$ output lines)
- **Stage 1 (s):** $n/p = 4/2 = 2$
- **Stage 2 (f):** $s/p = 2/2 = 1$
- **Total 1:2 DEMUXes Required:** $f + s = 1 + 2 = 3$



Implementation 2: 1x8 DEMUX using 1x4 and 1x2 DEMUX

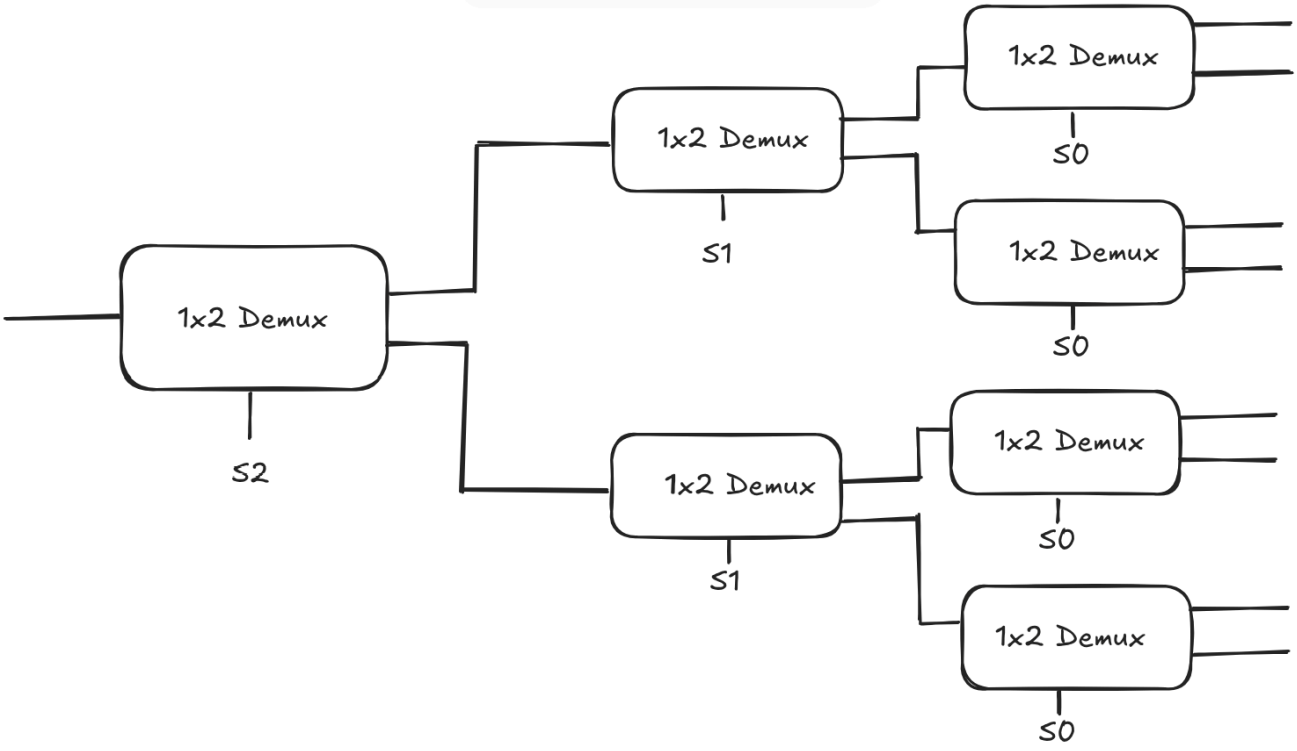
We need one 1×2 demultiplexer and two 1×4 demultiplexers.

- **Stage 1:** The primary input feeds into one 1×2 demultiplexer.
- **Stage 2:** The two outputs from the first stage feed into two 1×4 demultiplexers, providing the 8 final outputs.
- **Total Required:** One 1×2 DEMUX and two 1×4 DEMUXes.



Implementation 3: 1x8 DEMUX using only 1x2 DEMUX

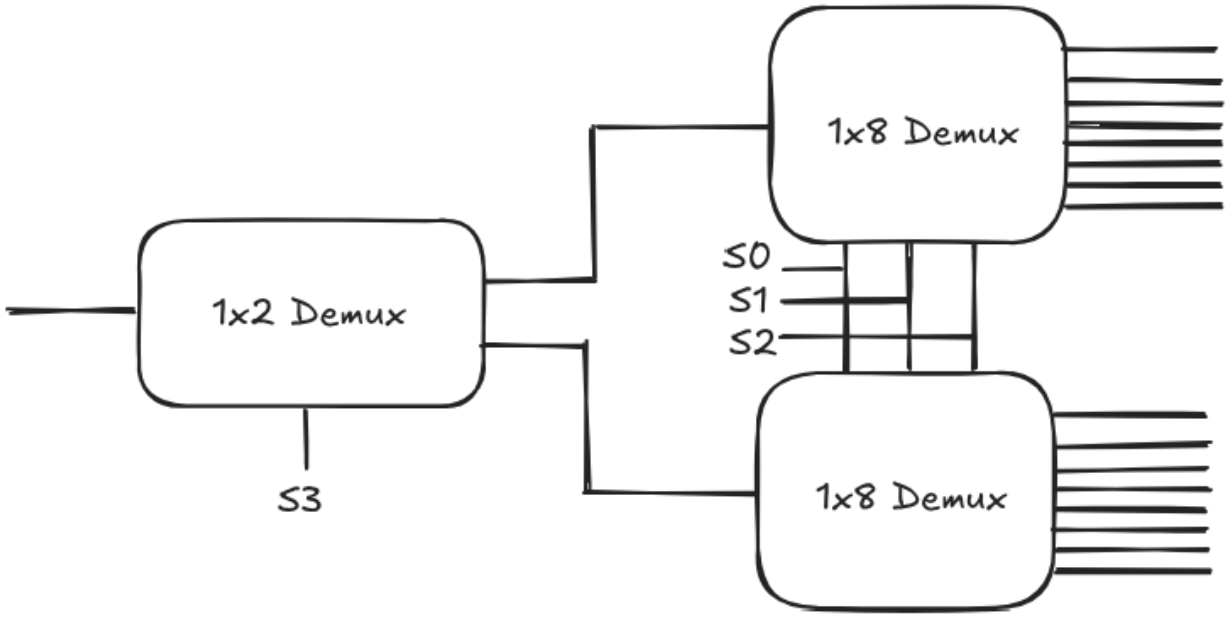
- **Required DEMUX:** 1:8 ($n = 8$ output lines)
- **Available DEMUX:** 1:2 ($p = 2$ output lines)
- **Stage 1 (t):** $n/p = 8/2 = 4$
- **Stage 2 (s):** $t/p = 4/2 = 2$
- **Stage 3 (f):** $s/p = 2/2 = 1$
- **Total 1:2 DEMUXes Required:** $f + s + t = 1 + 2 + 4 = 7$



Implementation 4: 1:16 DEMUX using 1:8 and 1:2 DEMUX

- **Required DEMUX:** 1:16 ($n = 16$ output lines)
- **Available DEMUXes:** 1:8 ($p_1 = 8$) and 1:2 ($p_2 = 2$)

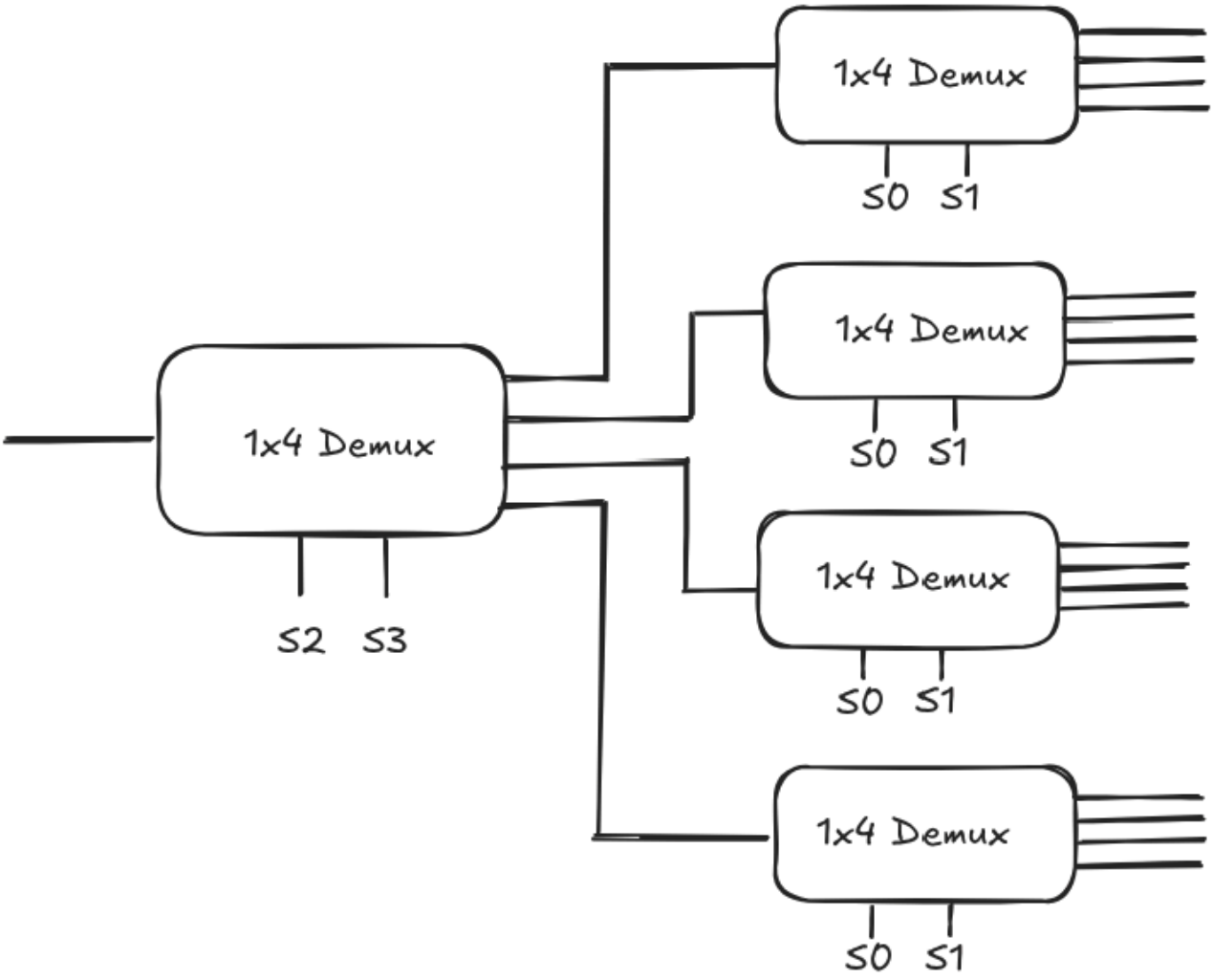
- **Stage 1:** The primary input feeds into one 1:2 DEMUX.
- **Stage 2:** The two outputs feed into two 1:8 DEMUXes.
- **Total Required:** One 1:2 DEMUX and two 1:8 DEMUXes.



Implementation 5: 1x16 DEMUX using only 1x4 DEMUX

- **Required DEMUX:** 1:16 ($n = 16$ output lines)
- **Available DEMUX:** 1:4 ($p = 4$ output lines)
- **Stage 1 (s):** $n/p = 16/4 = 4$
- **Stage 2 (f):** $s/p = 4/4 = 1$

- **Total 1:4 DEMUXes Required:** $f + s = 1 + 4 = 5$



Combinational Logic - Decoders

1. Overview of Encoders and Decoders

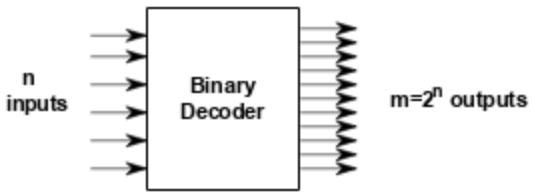
Encoders and decoders are foundational combinational modules used extensively in digital systems for data routing and compression.

- **Binary Decoders:** Convert an n-bit binary code into a single active output. They are built using AND/OR gate logic.
- **Binary Encoders:** Perform the reverse operation, converting one of 2^n inputs into an n -bit output. They are highly useful for data compression.

2. Binary Decoders

A decoder looks at its inputs, determines which binary number is present, and activates the one output corresponding to that number while keeping all other outputs inactive.

- In general, a decoder has N input lines and 2^N output lines.
- It is known as an n-to-m-line decoder (where $m = 2^n$).
- The basic decoding element is the AND gate, because it produces a logic 1 only when all inputs are 1.



The 2-to-4 Decoder

A 2-to-4 decoder has 2 inputs (A_1, A_0) and 4 outputs (D_3, D_2, D_1, D_0).

Truth Table:

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Boolean Expressions (Minterms):

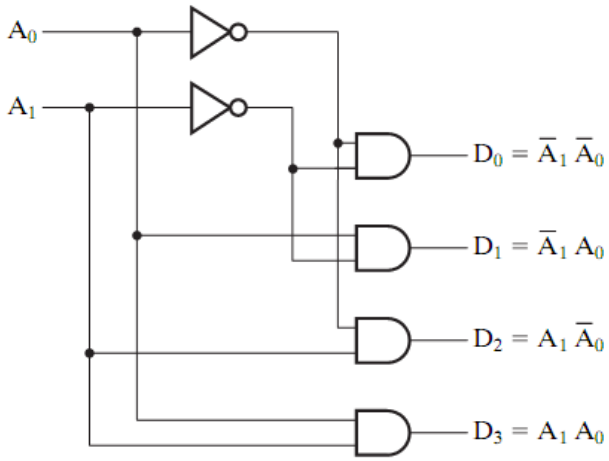
$$D_0 = \overline{A}_1 \overline{A}_0$$

$$D_1 = \overline{A}_1 A_0$$

$$D_2 = A_1 \overline{A}_0$$

$$D_3 = A_1 A_0$$

(Note: Each output represents exactly one 2-variable minterm).



Decoders with Enable (EN) Pin

Decoders often include an Enable pin.

- If $EN = 0$, the decoder is disabled and all outputs are forced to 0, regardless of the inputs.
- If $EN = 1$, the decoder functions normally according to its truth table.

Notice the use of "Don't Cares" (X) in the first row. Because $EN = 0$ kills the output, we don't care what A_1 and A_0 are.

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

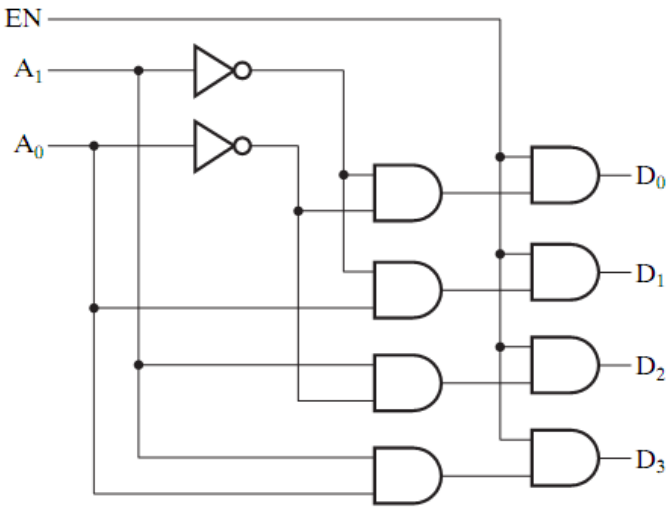
Boolean Expressions: Because the EN pin must be `1` for *any* output to trigger, it is simply ANDed (multiplied) into every single minterm equation.

$$D_0 = EN \cdot \overline{A}_1 \overline{A}_0$$

$$D_1 = EN \cdot \overline{A}_1 A_0$$

$$D_2 = EN \cdot A_1 \overline{A}_0$$

$$D_3 = EN \cdot A_1 A_0$$



The 3-to-8 Decoder

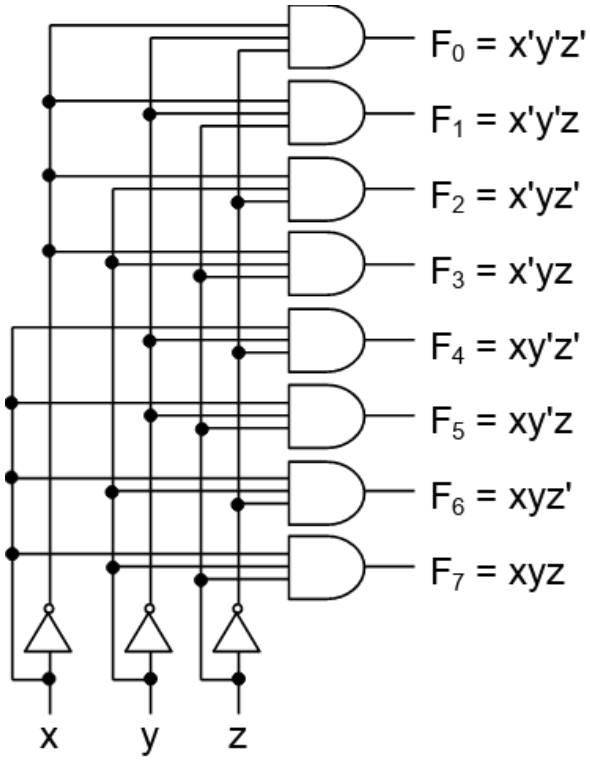
A 3-to-8 decoder takes 3 inputs (X, Y, Z) and generates 8 unique outputs (F_0 to F_7), corresponding to the 8 possible 3-variable minterms.

Truth Table:

X	Y	Z	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Boolean Expressions:

- $F_0 = X'Y'Z'$
- $F_1 = X'Y'Z$
- $F_2 = X'YZ'$
- $F_3 = X'YZ$
- $F_4 = XY'Z'$
- $F_5 = XY'Z$
- $F_6 = XYZ'$
- $F_7 = XYZ$


F0 = x'y'z'
F1 = x'y'z
F2 = x'yz'
F3 = x'yz
F4 = xy'z'
F5 = xy'z
F6 = xyz'
F7 = xyz
x
y
z

3. Implementing Logic Functions Using Decoders

Any combinational circuit with n inputs and m outputs can be implemented using an n-to-2^n decoder and m OR gates.

- The decoder generates all possible minterms of the input variables.
- The OR gate simply sums (adds together) the specific minterm outputs required by the Boolean function.
- This approach is highly efficient when a circuit has many outputs, and each function is expressed with few minterms.

*Example: Implementing a Full Adder

The equations for a Full Adder are:

- Sum: $S(x, y, z) = \Sigma m(1, 2, 4, 7)$
- Carry: $C(x, y, z) = \Sigma m(3, 5, 6, 7)$

**Step 1: The Truth Table

A Full Adder adds three binary bits together: x (Input 1), y (Input 2), and z (Carry-In). It produces two outputs: S (Sum) and C (Carry-Out). We evaluate all 8 possible input combinations to determine the outputs.

Truth Table:

Minterm	x	y	z	S (Sum)	C (Carry)
m_0	0	0	0	0	0
m_1	0	0	1	1	0
m_2	0	1	0	1	0
m_3	0	1	1	0	1
m_4	1	0	0	1	0
m_5	1	0	1	0	1
m_6	1	1	0	0	1
m_7	1	1	1	1	1

**Step 2: Minterm Derivation

To use a decoder, we must express the outputs in standard **Sum of Minterms (SOP)** format. We do this by looking at the truth table and writing down the minterm index (m_i) everywhere an output is logic `1`.

For the Sum (S):

The output S is `1` at rows 1, 2, 4, and 7.

$$S(x, y, z) = \Sigma m(1, 2, 4, 7)$$

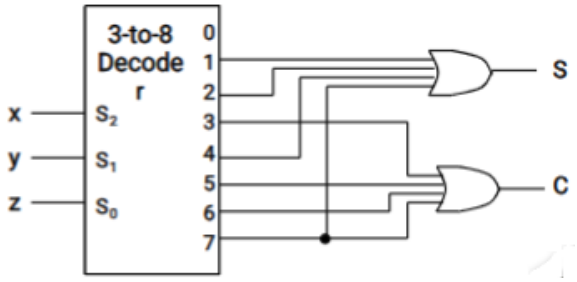
Expanded: $S = \overline{x}\overline{y}z + \overline{x}y\overline{z} + x\overline{y}\overline{z} + xyz$

For the Carry (C):

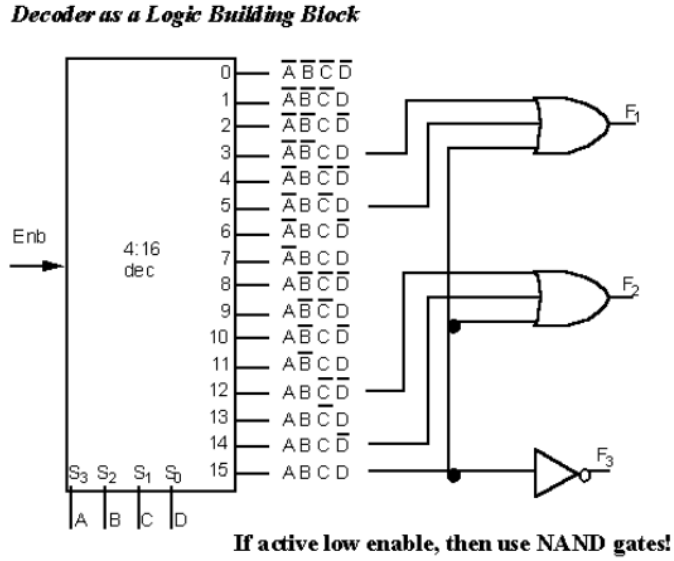
The output C is `1` at rows 3, 5, 6, and 7.

$$C(x, y, z) = \Sigma m(3, 5, 6, 7)$$

Expanded: $C = \overline{x}yz + x\overline{y}z + xy\overline{z} + xyz$



Example 2:



- **F1= A'B'CD + A'BC'D + ABCD**
- **solve the exercise for F_2 and F_3.

Solving for F_2

By tracing the three input wires of the F_2 OR gate backward to the 4:16 decoder outputs, we can identify its specific minterms:

- The top wire connects to **Pin 12**.
- The middle wire connects to **Pin 13**.
- The bottom wire connects to **Pin 14**.

Now, we just convert those pin numbers into their 4-variable Boolean equivalents based on the inputs A, B, C, and D (where A is the Most Significant Bit):

- Pin 12 (Binary `1100`) = $AB\overline{C}\,\overline{D}$
- Pin 13 (Binary `1101`) = $AB\overline{C}D$
- Pin 14 (Binary `1110`) = $ABC\overline{D}$

Since they feed into an OR gate, we add them together for the final expression:

$$F_2 = AB\overline{C}\,\overline{D} + AB\overline{C}D + ABC\overline{D}$$

**Solving for F_3

By tracing the single wire from the F_3 NOT gate (inverter) backward, we see it connects to the junction dot on **Pin 15**.

- Pin 15 (Binary `1111`) = $ABCD$

Because this signal passes through an inverter before reaching the output, the equation is the complement (NOT) of that minterm:

$$F_3 = (ABCD)'$$

If your professor prefers fully expanded expressions, you can apply De Morgan's Law to distribute the inversion:

$$F_3 = \overline{A} + \overline{B} + \overline{C} + \overline{D}$$

4. Decoder Expansion (Cascading)

You can build larger decoders using smaller ones by utilizing the Enable pin.

Example: Building a 4-to-16 Decoder using two 3-to-8 Decoders

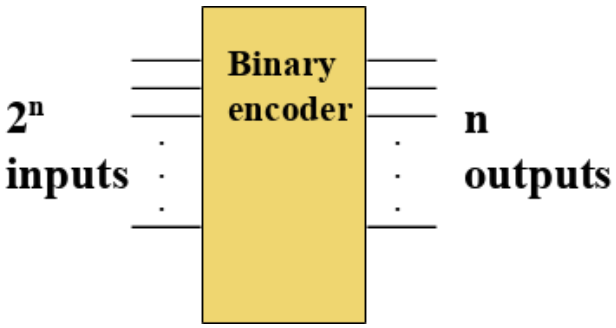
- The 3 lower-order input bits (x, y, z) are wired into the data inputs of **both** 3-to-8 decoders.
- The 1 highest-order input bit (w) acts as the Master Select and is wired to the Enable pins.
- The w input is inverted before going into the top decoder. When $w = 0$, the top decoder is enabled (generating minterms 0-7), and the bottom is disabled.
- When $w = 1$, the enable conditions flip. The bottom decoder turns on and generates minterms 8-15.

Combinational Logic - Encoders

5. Binary Encoders

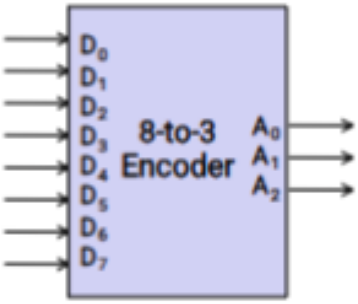
Encoding is the exact opposite of decoding. An encoder has 2^n input lines and n output lines.

- It assumes that **exactly one** input line is logic 1 at any given time.
- It outputs the binary code corresponding to the physical position (subscript number) of the active input.



8-to-3 Encoder Truth Table

- 2^3 = 8 inputs, 3 outputs
- one input =1, others = 0's
- Each input generate unique binary code



D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

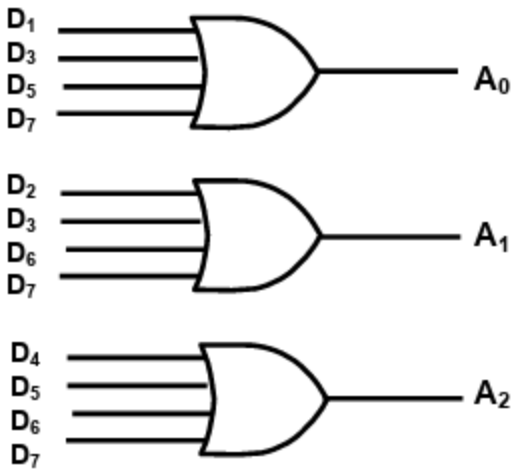
Derived Boolean Expressions:

By looking at the columns for A_0, A_1, and A_2, you can derive the equations by ORing together the specific D inputs that produce a `1` in that respective output column:

$$A_0 = D_1 + D_3 + D_5 + D_7$$

$$A_1 = D_2 + D_3 + D_6 + D_7$$

$$A_2 = D_4 + D_5 + D_6 + D_7$$



Standard encoders have two fatal flaws:

1. **Multiple Active Inputs:** If two inputs are active simultaneously (e.g., $D_3 = 1$ and $D_6 = 1$), the output is garbled (it outputs `111`, which is incorrect for both).
2. **All Zero Inputs:** If all inputs are 0, the output is `000`. However, `000` is also the output when D_0 is active! The system cannot tell the difference between "Input 0 is active" and "The machine is turned off".

6. The Priority Encoder

The Priority Encoder is designed to solve the two limitations of standard encoders.

1. If multiple inputs are active, it only pays attention to the input with the **highest priority** (the highest subscript number).
2. It includes a third output called the **Valid Bit (V)**. If any input is active, $V = 1$. If all inputs are zero, $V = 0$.

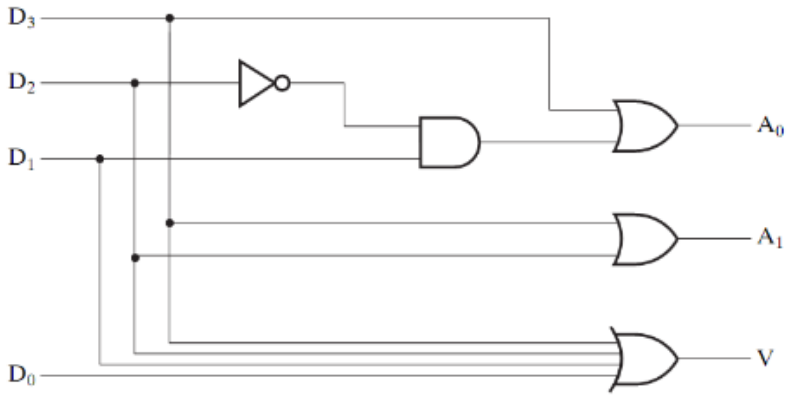
4-to-2 Priority Encoder (Condensed Truth Table): Because we ignore lower-order bits when a high-priority bit is active, we use "Don't Cares" (X) to condense a 16-row truth table down to just 5 rows.

D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1

(Observe how D_3 overrides everything below it).

K-Map Simplified Equations: Applying the condensed truth table to Karnaugh Maps yields the minimized hardware equations:

- **Output A0:** $A_0 = D_3 + D_1\overline{D_2}$
- **Output A1:** $A_1 = D_2 + D_3$
- **Valid Bit:** $V = D_0 + D_1 + D_2 + D_3$



Sequential Circuits and Latches

1. Introduction to Sequential Circuits

Combinational circuits only depend on the current input values to produce outputs. Sequential circuits, however, are digital circuits that store and use previous state information to determine their next state.

- Outputs in a sequential circuit are a function of both the current inputs and the present state of the storage elements.
- The general purpose of these circuits is to store information for an indefinite amount of time.

Applications:

Sequential circuits are commonly used in digital systems to implement:

- State machines
- Timers
- Counters
- Memory elements (implemented using flip-flops, which maintain their binary state even when inputs change)

2. Types of Sequential Circuits

Sequential circuits are divided into two primary categories based on how they handle time and state changes.

Synchronous vs. Asynchronous Circuits

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Clock Signal	Uses a clock to synchronize changes.	Does not use a clock; changes occur immediately with input changes.
Operation	All changes happen at specific time intervals (clock pulses).	Changes occur as soon as inputs change, without waiting for a clock.
Speed	Slower due to clock dependency.	Faster because it reacts immediately to input changes.

Feature	Synchronous Sequential Circuit	Asynchronous Sequential Circuit
Design Complexity	Easier to design and analyze because changes happen in sync.	More complex due to potential timing issues (hazards, glitches).
Reliability	More reliable and predictable.	Less reliable due to possible race conditions.

3. Introduction to Latches and Flip-Flops

Synchronous sequential circuits that use clock pulses as inputs for their storage elements are called **clocked sequential circuits**. The simplest form of storage element used in these circuits is the flip-flop.

- **Flip-Flop:** A binary storage device capable of storing one bit of information and having specific timing characteristics.
- **Latches:** The most basic storage elements from which flip-flops are usually constructed.
- Latches store a single bit of information and hold its value until it is updated by new input signals.
- They act as temporary storage elements and can be implemented using various digital logic gates, such as AND, OR, NOT, NAND, and NOR gates.

4. The S-R (Set-Reset) Latch

Set-Reset latches are the simplest form of latches and form the basic building blocks for all other types of flip-flops.

- It is implemented using two inputs: S (Set) and R (Reset).
- The S input sets the output to logic 1, while the R input resets the output to logic 0.
- They are also known as "preset" and "clear" states.
- A basic SR Latch consists of 2 cross-coupled NOR gates or 2 cross-coupled NAND gates, producing 2 outputs: Q and Q'.

Building SR Latch (Using NAND Gate):

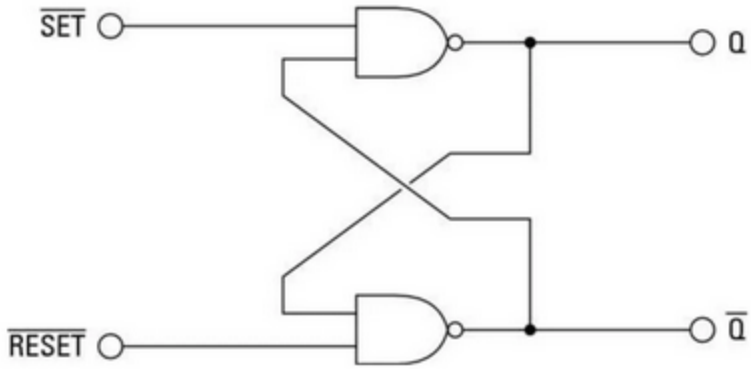
When built with NAND gates, the SR Latch becomes an Active-Low device (often denoted as $\overline{S}$-$\overline{R}$ Latch). This means that a logic 0 triggers the action, and logic 1 is the neutral resting state.

Logic Rules:

- **Hold State ($S' = 1, R' = 1$):** Both inputs are inactive (resting at High). The latch holds its previous value.

- **Set State ($S' = 0, R' = 1$):** Driving S' low forces Q to 1, which feeds back to make $Q' = 0$.
- **Reset State ($S' = 1, R' = 0$):** Driving R' low forces Q' to 1, which feeds back to make $Q = 0$.
- **Invalid State ($S' = 0, R' = 0$):** Both inputs dropping to 0 simultaneously forces both NAND gates to output a 1. Therefore, $Q = 1$ and $Q' = 1$, which violates the inverse rule and creates an undefined state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Invalid (ndefined)
0	1	1 (Set)
1	0	0 (Reset)
1	1	Hold (No Change)



Building SR Latch (Using NOR Gate):
Truth Table When built with NOR gates, the SR Latch is an Active-High device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

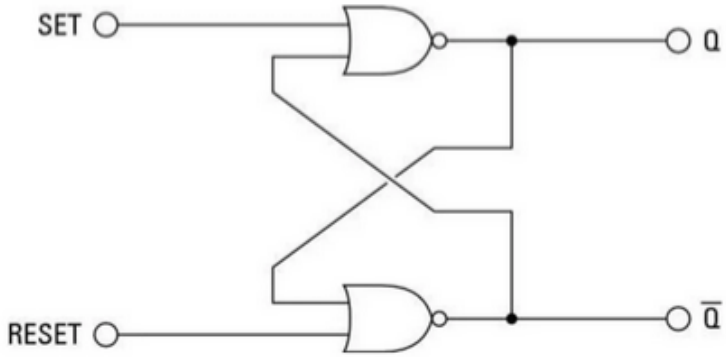
Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.(Using NOR Gate):
  When built with NOR gates, the SR Latch is an Active-High** device. This means that a logic 1 triggers the Set or Reset action, while logic 0 is the neutral/resting state.

Logic Rules:

- **Hold State ($S = 0, R = 0$):** Both inputs are inactive. The cross-coupled feedback loop maintains whatever value was previously stored in Q.
- **Set State ($S = 1, R = 0$):** The S input forces Q' to 0, which feeds back to make $Q = 1$.
- **Reset State ($S = 0, R = 1$):** The R input forces Q to 0, which feeds back to make $Q' = 1$.
- **Invalid State ($S = 1, R = 1$):** Both inputs attempt to force a zero. This forces both Q and Q' to equal 0 simultaneously. This violates the rule that Q and Q' must be inverses, making it an undefined/invalid state.
  Truth Table:

S'	R'	Q (Next State)
0	0	Hold (No Change)
0	1	0 (Reset)
1	0	1 (Set)
1	1	Invalid (ndefined)



5. S-R Latch with Control Input

Latches can be modified by adding a control input (C), which acts as an enable signal or clock. These controlled latches are used to construct more advanced flip-flops.

Logic Rules:

- When $C = 0$: The control input disables the circuit. The state of the output does not change, regardless of the values of S and R.
- When $C = 1$: The latch operates normally.
- When $C = 1$ and $S = R = 0$: The state holds (no change).
- When $C = 1$ and $S = R = 1$: An undefined state occurs.

Truth Table (SR Latch with Control):

C	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$ Reset state
1	1	0	$Q = 1$ Set state
1	1	1	ndefined

6. The D (Data) Latch

The D Latch (also called a "Transparent Latch") was designed specifically to eliminate the undesirable undefined state present in the SR latch.

It simplifies the architecture down to only two inputs: D (Data) and C (Control). Internally, the D input is wired directly to the S terminal, and passed through an inverter to the R terminal. This guarantees that S and R can never be 1 at the same time.

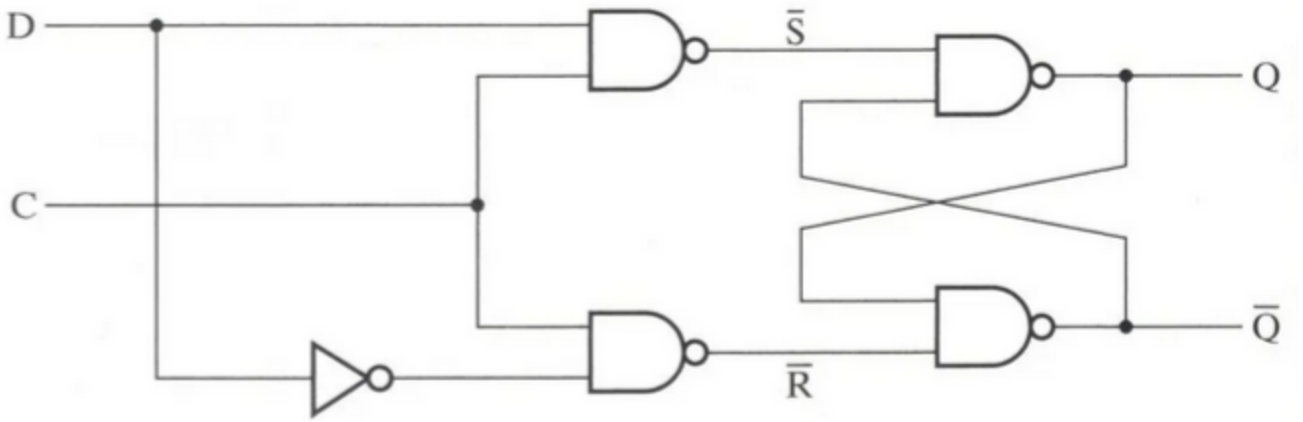
Logic Rules:

- The D input is only sampled when $C = 1$.
- If $D = 1$ (and $C = 1$), the Q output goes to 1 (Set state).
- If $D = 0$ (and $C = 1$), the Q output goes to 0 (Reset state).

Truth Table (D Latch):

C	D	Next state of Q
0	X	No change (Hold)
1	0	$Q = 0$ Reset state
1	1	$Q = 1$ Set state

The D Latch Circuit Diagram

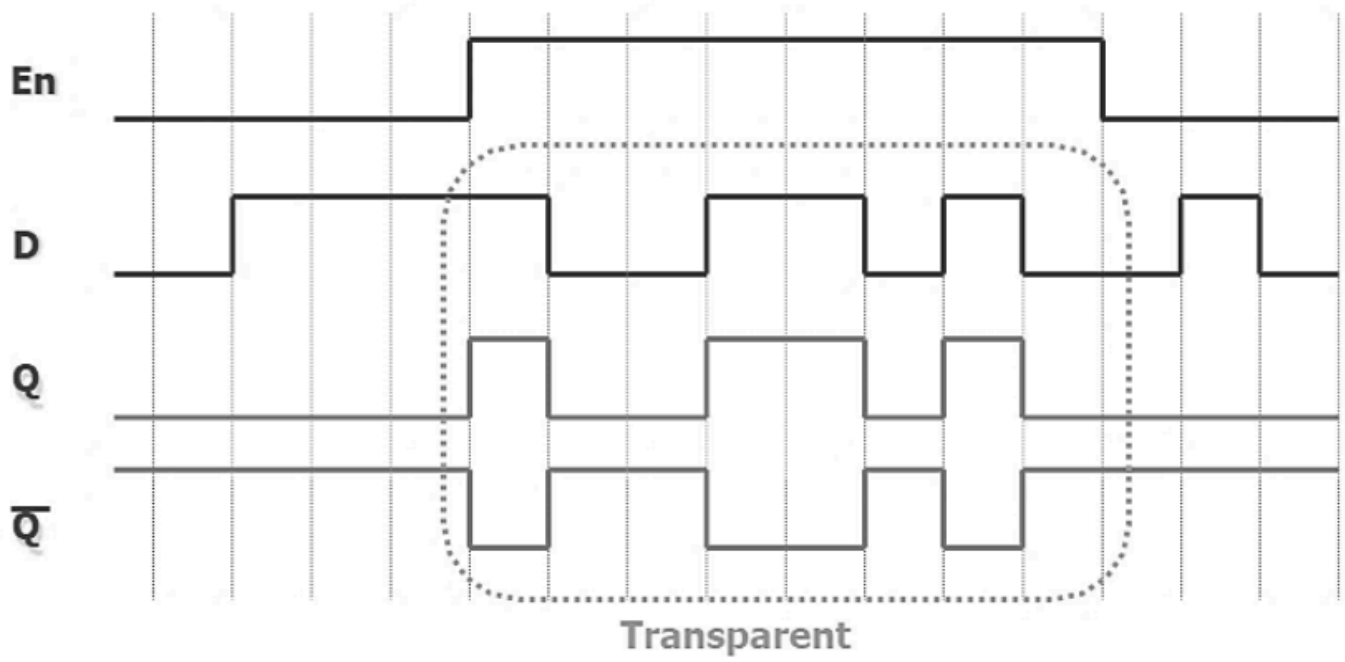


Transparent Latch

1. The D Latch as a "Transparent Latch"

As discussed previously, a D Latch uses a Control input (En or C) to sample data.

- When $En = 0$, the latch holds its value. It acts as a closed **Storage Cell**.
- When $En = 1$, the latch becomes **transparent**. This means the output Q instantly tracks any changes made to the input D. It acts like a straight wire.



The Clock Cycle

In synchronous digital systems, the Enable signal is driven by a continuous Clock Cycle.

- A clock cycle consists of one "up" phase (High/Enable) and one "down" phase (Low/Disable).
- The frequency is the number of these complete cycles per second.



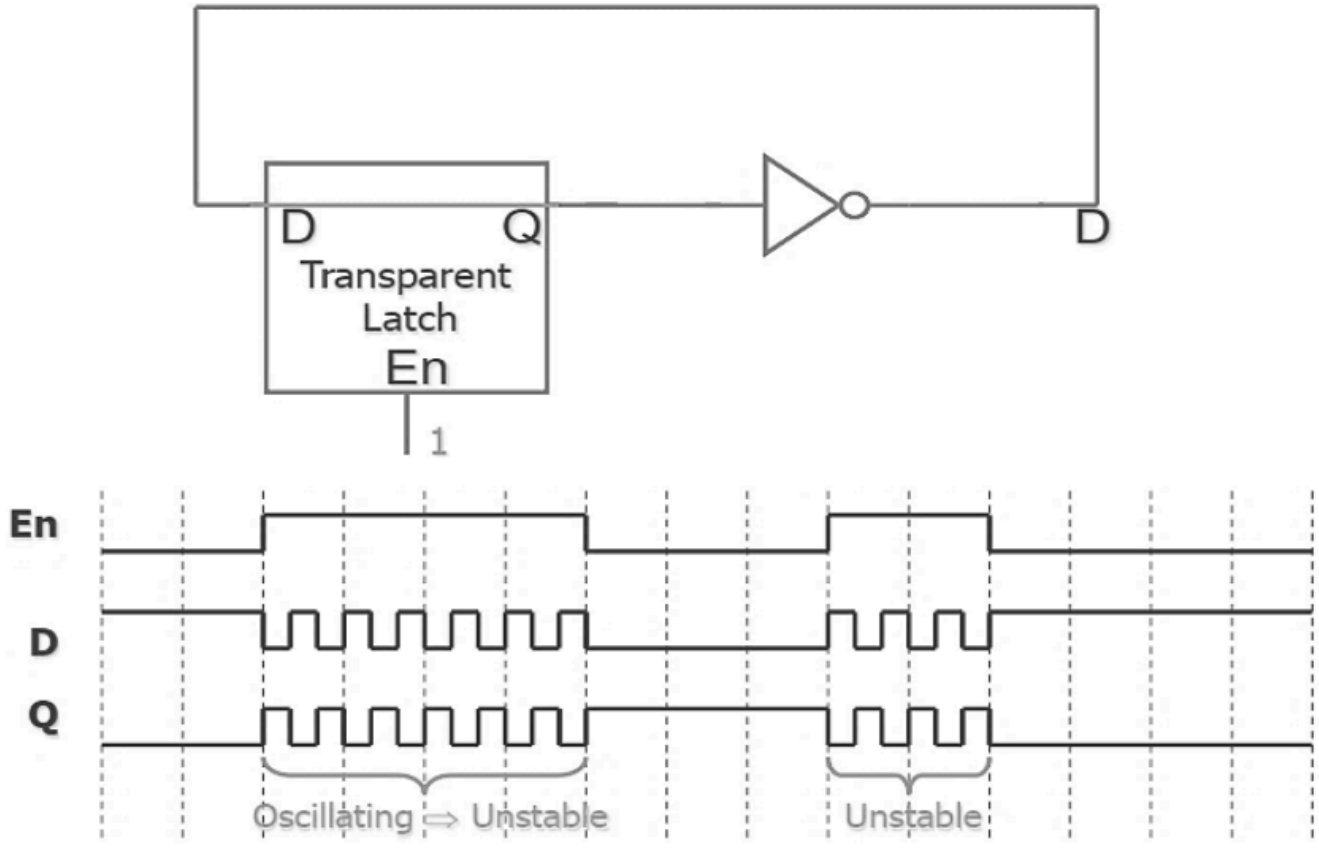
2. The Problem of Transparency

The defining feature of a D-Latch—its transparency during the High clock pulse—is also its biggest flaw in complex circuits.

The Setup: Imagine a scenario where the output Q of a Transparent D-Latch feeds into a block of combinational logic, and the result of that logic loops back to feed the input D of that same latch.

The Issue (Feedback Oscillation):

1. When the clock pulse is Low ($En = 0$), the latch works perfectly, holding its 1-bit value.
2. However, when the clock pulse goes High ($En = 1$), the latch becomes a transparent wire.
3. If the combinational logic is an **Inverter (NOT gate)**, the output Q is inverted and instantly fed back into D.
4. Because the latch is transparent, Q immediately changes to match the new D, which is then inverted again, fed back into D again, and so on.
5. This creates an **unstable oscillation loop** ($0 \to 1 \to 0 \to 1$) that continues endlessly as long as the clock remains High.
6. When the clock finally drops back to Low ($En = 0$), the latch will snap shut, but it may trap erroneous or random data depending on exactly when the clock fell during the oscillation.



3. The Solution: Flip-Flops

To solve the instability caused by transparency, engineers must completely isolate the input from the output during a clock cycle. This is achieved by creating a **Flip-Flop**.

What is a Flip-Flop?

- A Flip-Flop is essentially a combination of two separate latches wired together in series.
- The first latch is called the **Master** (often a D Latch), and the second is called the **Slave** (which can be an SR Latch or another D Latch).
- The clock signal driving the Slave is inverted from the clock signal driving the Master.

How it eliminates transparency: Because the clocks are inverted, only one latch can ever be open (transparent) at a given time.

- When the clock is High, the Master is open and accepts new input data, but the Slave is closed, preventing that data from reaching the final output.
- When the clock goes Low, the Master closes (locking in the data), and the Slave opens to safely pass that locked data to the final output.
- The input (D) is never directly connected to the final output (Q), completely eliminating the feedback oscillation problem.

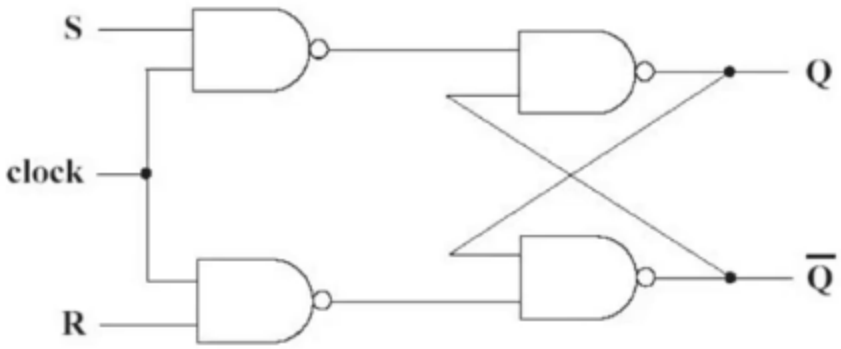
Sequential Circuits - The Clocked SR Flip-Flop

1. Clocked SR Flip-Flop (NAND Implementation)

To control when the SR Latch changes state, we add a clock (or enable) signal. By adding two "steering" NAND gates to the front of a basic active-low NAND latch, we create a Clocked SR Flip-Flop (often called a Gated SR Latch).

- The clock input (`Clk`) acts as a gatekeeper.
- When `Clk = 0` , the steering NAND gates output `1` regardless of S and R. Because the core latch is active-low, a `1` tells it to hold its state.
- When `Clk = 1` , the S and R inputs are inverted and passed through to the core latch, allowing it to function normally.

Circuit Diagram

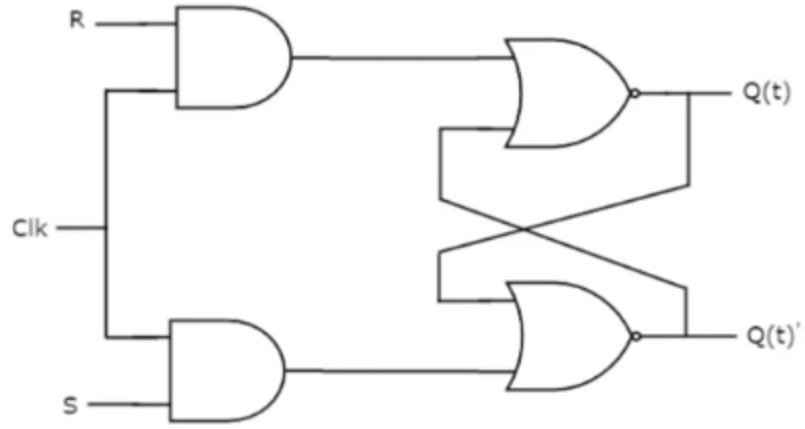


2. Clocked SR Flip-Flop (NOR Implementation)

To build a Clocked SR Flip-Flop around a NOR-gate core, we must use **AND gates** as the steering logic instead of NAND gates.

- The core NOR Latch is naturally an Active-High device (it requires a `1` to change state).
- When `Clk = 0` , the steering AND gates output `0` regardless of the S and R inputs. Supplying `0, 0` to a NOR latch tells it to Hold its state.
- When `Clk = 1` , the S and R inputs pass directly through the AND gates into the core latch without being inverted, triggering the Set or Reset actions normally.

Circuit Diagram



2. Fundamental Flip-Flop Tables

To fully analyze a sequential circuit, we use three specific types of tables. Each serves a different engineering purpose.

A. The Truth Table (Function Table)

The Truth Table defines the overarching behavior of the flip-flop based on the current inputs and the clock.

Clk	S	R	Q_{n+1} (Next State)
0	X	X	Q_n (Hold)
1	0	0	Q_n (Hold)
1	0	1	0 (Reset)
1	1	0	1 (Set)
1	1	1	Invalid

B. The Characteristic Table

The Characteristic Table is used for **analysis**. It explicitly details what the next state (Q_{n+1}) will be for every possible combination of the current state (Q_n) and the current inputs (S, R).

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X (Don't Care/Invalid)
1	1	1	X (Don't Care/Invalid)

C. The Excitation Table

The Excitation Table is used for **design**. If you know what state you are currently in (Q_n) and you know what state you want to go to (Q_{n+1}), this table tells you what inputs (S, R) you must apply to make that transition happen.

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

(Note: X represents a "Don't Care" condition. For example, to transition from $0 \rightarrow 0$, S must be 0, but R can be either 0 or 1, so it is marked as X).

3. The Characteristic Equation

D. K-Map for the Characteristic Equation

To find the equation for the Next State (Q_{n+1}), we plot the outputs from the Characteristic Table onto a 3-variable K-Map based on the inputs S, R, and Q_n.

Note: The Invalid states where $S = 1$ and $R = 1$ are plotted as "Don't Cares" (X). This is a crucial engineering trick, as it allows us to use those cells to make larger groups and simplify the final hardware equation.

$S \backslash RQ_n$	00	01	11	10
0	0	1	0	0
1	1	1	X	X

Grouping the K-Map:

To get the most simplified equation, we circle the largest possible groups of `1`s (incorporating `X`s if they help make the group larger).

1. **Group 1 (The Horizontal Row):** Look at the entire bottom row ($S = 1$). It contains `1, 1, X, X`. We can group all 4 of these cells together. Because this group spans across all possible values of R and Q_n, those variables are eliminated.
   - **Resulting Term:** S
2. **Group 2 (The Vertical Column):** Look at the column where $R = 0$ and $Q_n = 1$ (the `01` column). It contains a `1` in the top row and a `1` in the bottom row. We group these 2 cells vertically. Because this group spans across $S = 0$ and $S = 1$, the S variable is eliminated. The constant variables for this column are $R = 0$ (which is R') and $Q_n = 1$ (which is Q_n).

- **Resulting Term:** $R'Q_n$

The Final Derived Equation:

By ORing (adding) the terms from our two groups, we get the minimized Characteristic Equation for the SR Flip-Flop:

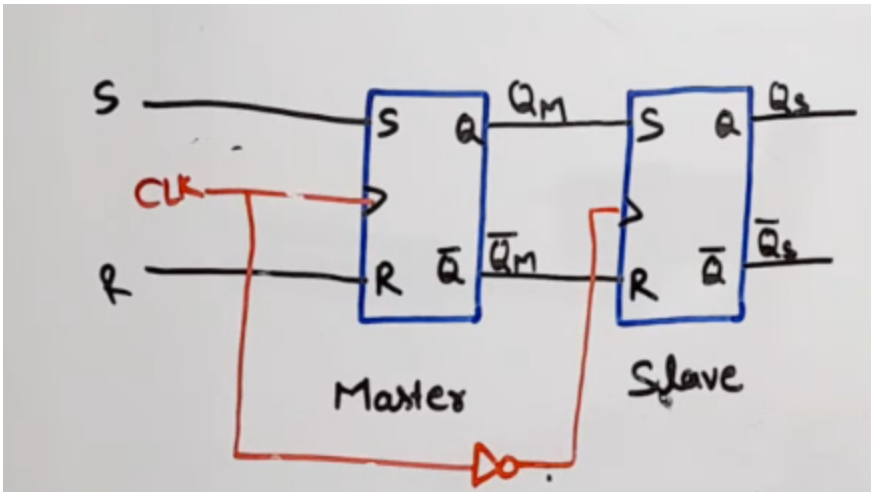
$$Q_{n+1} = S + R'Q_n$$

The Master-Slave SR Flip-Flop

n## 1. The Transparency Problem and the Master-Slave Solution

As discussed previously, a standard Clocked SR Latch is **level-sensitive** (transparent). As long as the clock remains High (`1`), any changes to the S and R inputs will immediately pass through and change the output. In complex, looped circuits, this transparency causes unstable feedback oscillations.

To solve this, we must completely isolate the inputs from the final outputs. We do this by chaining two Clocked SR Latches together in a **Master-Slave** configuration.



The Architecture

- **The Master:** The first gated latch. It receives the external S and R inputs and the raw Clock signal.
- **The Slave:** The second gated latch. It receives the outputs of the Master (Q_m and $\overline{Q}_m$) as its inputs. Crucially, **the clock signal is inverted** before it reaches the Slave.

2. Principle of Operation

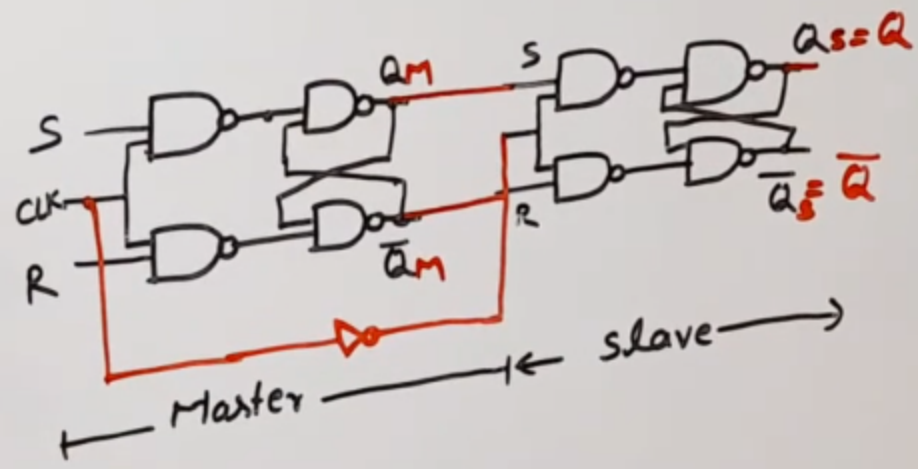
Because the clock is inverted between the two stages, only one latch can be "open" (transparent) at any given time.

1. **When Clock is High ($Clk = 1$):** The Master latch is enabled. It accepts the S and R inputs and updates its internal state (Q_m).
   - The inverted clock going to the Slave is `0`. The Slave is disabled, locking the final output (Q_s) safely in its previous state. The circuit is actively loading data, but the output is shielded.
2. **When Clock is Low ($Clk = 0$):**

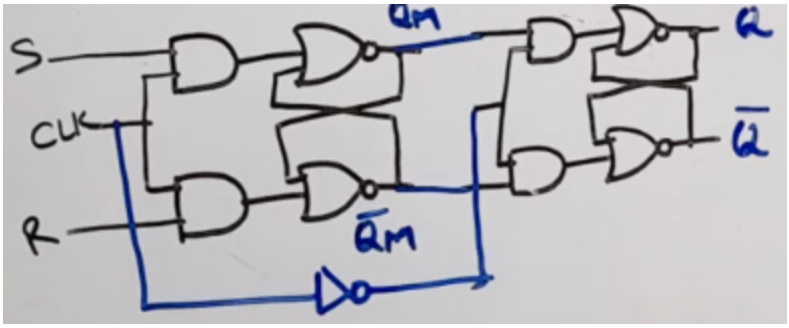
- The Master latch is disabled. It stops accepting external inputs and locks its current Q_m value.
- The inverted clock going to the Slave becomes `1`. The Slave wakes up, reads the locked Q_m value from the Master, and updates the final output (Q_s).

This mechanism creates **Edge-Triggered** behavior. From the outside, it appears that the final output (Q_s) only ever changes at the exact moment the clock falls from `1` to `0` (the negative edge).

Circuit Diagram (NAND Implementation)



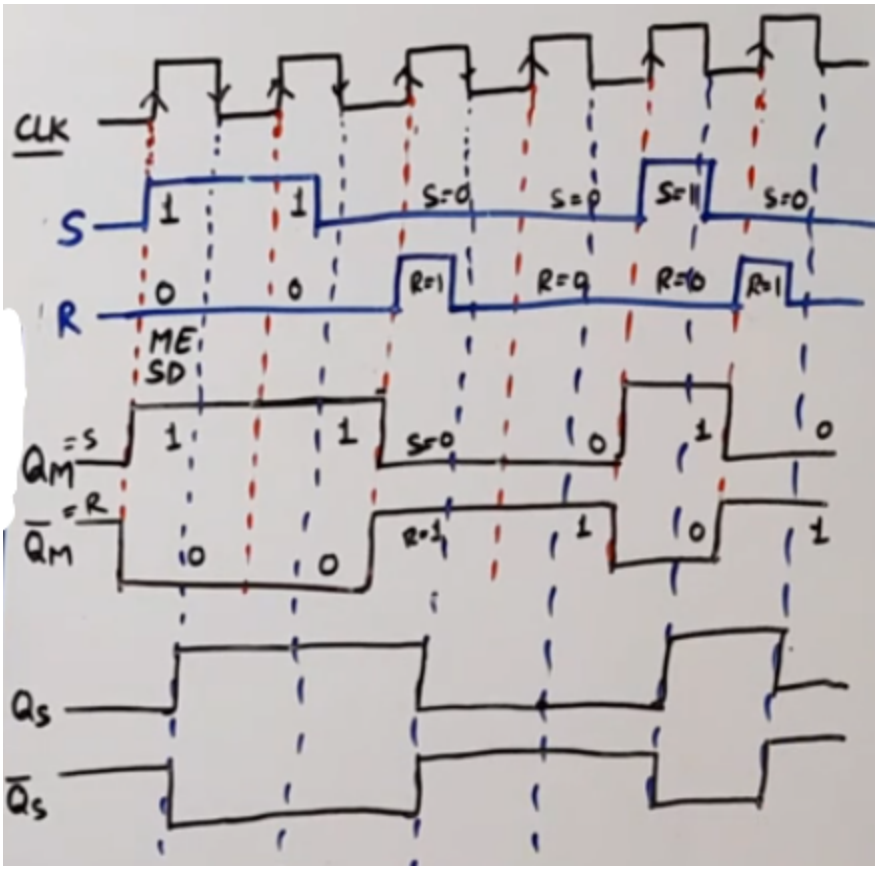
Circuit Diagram (NOR Implementation)



3. Timing Diagram Analysis

To truly understand a Master-Slave flip-flop, you must analyze its waveforms over time.

- **S and R:** The user inputs.
- **Q_m (Master Output):** Tracks the inputs *only* while the clock is High.
- **Q_s (Slave / Final Output):** Only updates to match Q_m exactly at the falling edge of the clock (when it transitions High $\rightarrow$ Low).



This ensures that any chaotic signals or rapid switching on the S and R lines during a clock cycle are safely absorbed by the Master and never accidentally leak to the rest of the computer system.

Sequential Circuits - The JK Flip-Flop

1. SR vs. JK Behavior

The primary purpose of the JK Latch is to resolve the undefined/invalid state of the SR Latch. By crisscrossing the final outputs back to the input gates, the circuit uses its own current state to determine how to handle a `1, 1` input.

As shown , the function tables compare directly:

SR Latch Function:

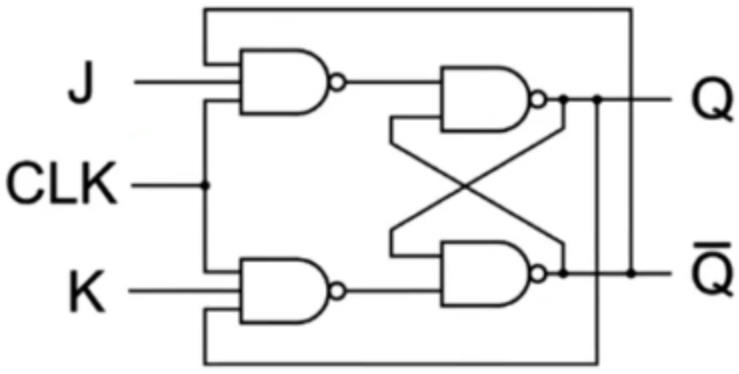
S	R	Q
0	0	Hold
0	1	0
1	0	1
1	1	Invalid

JK Latch Function:

J	K	Q_n
0	0	Hold
0	1	0
1	0	1
1	1	Toggle

2. Circuit Diagram (NAND Implementation)

This is the schematic you drew at the top of the page. Notice how it is essentially a clocked SR latch, but with the outputs Q and $\overline{Q}$ fed all the way back into the front steering NAND gates.



3. The Analytical Tables

These are the tables which explicitly map out the math behind the JK logic.

A. Characteristic Table

This table defines exactly what the next state (Q_{n+1}) will be based on the current inputs (J, K) and the current state (Q_n).

J	K	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

(Notice the last two rows: When $J = 1$ and $K = 1$, the next state is always the exact opposite of the current state Q_n. This is the Toggle feature in action).

B. Excitation Table

This table is used backward for circuit design. It tells you what inputs to apply to J and K if you want to force a specific state transition.

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

4. K-Map Derivation for the Characteristic Equation

To find the equation for the Next State (Q_{n+1}), we plot the outputs from the JK Characteristic Table onto a 3-variable K-Map based on the inputs J, K, and Q_n.

Unlike the SR Flip-Flop, the JK Flip-Flop has no "Invalid" states, meaning there are absolutely no "Don't Cares" (X) to use as wildcards. We must rely entirely on the definitive `1` s.

$J\backslash KQ_n$	00	01	11	10
0	0	1	0	0
1	1	1	0	1

Grouping the K-Map:

We have four `1`s on the board, and we can group them into two distinct pairs (loops of 2).

1. **Group 1 (The Bottom Corners):** Look at the bottom row ($J = 1$). There is a `1` in the `00` column and a `1` in the `10` column. Because K-Maps wrap around the edges, we can group these two corners together.
   - Across these two cells, K changes from 0 to 1, so it is eliminated.
   - The constant variables are $J = 1$ and $Q_n = 0$.
   - **Resulting Term:** $J\overline{Q}_n$
2. **Group 2 (The Vertical Column):** Look at the column where $K = 0$ and $Q_n = 1$ (the `01` column). There is a `1` in the top row and a `1` in the bottom row. We group these vertically.
   - Across these two cells, J changes from 0 to 1, so it is eliminated.
   - The constant variables are $K = 0$ and $Q_n = 1$.
   - **Resulting Term:** $\overline{K}Q_n$

The Final Derived Equation:

By ORing these two groups together, we get the minimized Characteristic Equation that governs the JK Flip-Flop:

$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

The Master-Slave JK Flip-Flop

1. The Race-Around Condition

As discussed, a single-stage, level-triggered JK Latch is mathematically unstable. If you feed the outputs (Q and Q') directly back into the input steering gates while the clock is High (1) and inputs are $J = 1, K = 1$, the circuit will enter a **"Race-Around Condition."** Because it takes only a few nanoseconds for electrons to pass through the gates, the flip-flop will toggle, feed its new toggled state back to the inputs, toggle again, and continue oscillating wildly for as long as the clock remains High.

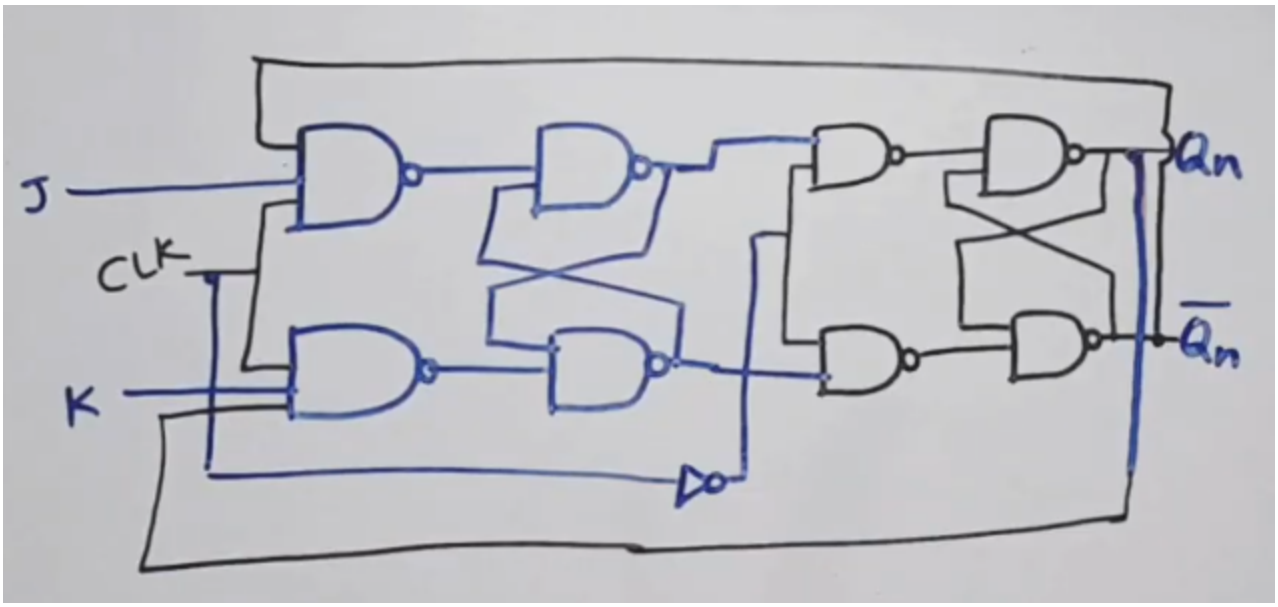
2. The Master-Slave Solution

To solve this, we break the feedback loop using a Master-Slave configuration.

- **The Master:** Receives J, K, the standard Clock signal, and the feedback from the Slave. It catches the new data when the clock is High.
- **The Slave:** Receives the outputs of the Master and an inverted Clock signal. It only updates the final output when the clock falls to Low.

Because the final output (which drives the feedback loop) only changes when the Master is safely locked, the circuit can only toggle exactly once per clock cycle.

Circuit Diagram (NAND Implementation)



3. Truth Table (Function Table)

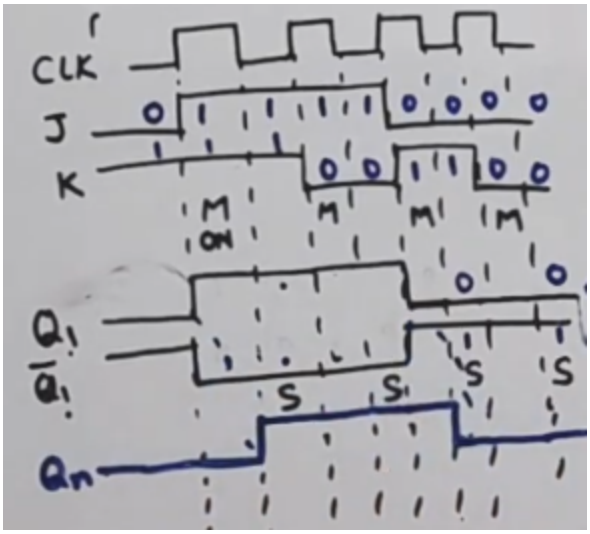
To mathematically represent edge-triggered behavior, we use the falling-edge arrow ($\downarrow$) for the clock. If the clock is steady at 0, 1, or rising, the outputs ignore J and K and hold their state.

Clk	J	K	Q_{n+1} (Next State)
0	X	X	Q_n (Hold)
1	X	X	Q_n (Hold)
$\uparrow$	X	X	Q_n (Hold)
$\downarrow$	0	0	Q_n (Hold)
$\downarrow$	0	1	0 (Reset)
$\downarrow$	1	0	1 (Set)
$\downarrow$	1	1	$\overline{Q}_n$ (Toggle)

4. Timing Diagram Analysis

Timing diagram perfectly illustrates how the Master and Slave isolate the signals.

- **Q_m (Master Output):** Q_m changes while the Clock is High (transparent phase for the Master).
- **Q_s (Slave / Final Output):** Q_s stays perfectly flat until the exact moment the Clock falls (indicated by the dashed vertical lines dropping from the Clock signal).



To clearly understand *why* the feedback loop allows it to toggle safely without racing around, experiment with this Master-Slave JK timing simulator. Pay close attention to what the Master does when you hold `J=1` and `K=1`.

Sequential Circuits - Single-Input Flip-Flops (T & D)

While SR and JK flip-flops offer multiple control inputs, we often only need a circuit to do one of two specific things: act as a pure memory delay (D) or act as a frequency toggler (T). We can create these single-input flip-flops by mathematically modifying our existing dual-input circuits.

---

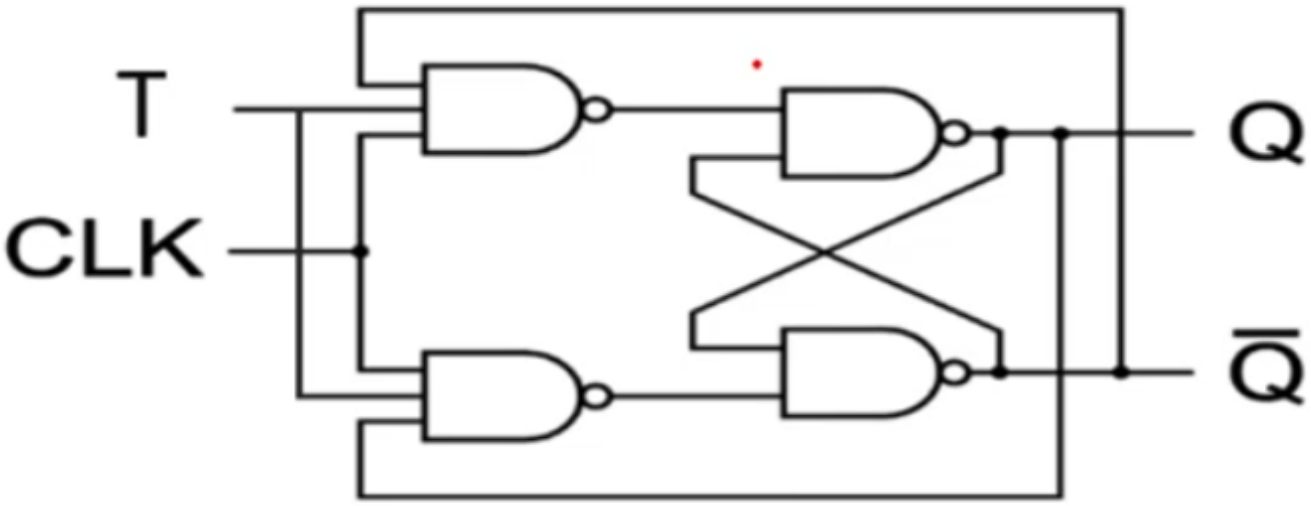
1. The T (Toggle) Flip-Flop

The T flip-flop is created by tying the J and K inputs of a JK Flip-Flop together into a single wire called T.

- If $T = 0$, both J and K receive `0`, meaning the circuit Holds its state.
- If $T = 1$, both J and K receive `1`, meaning the circuit Toggles its state on the clock edge.

Because holding $T = 1$ causes the output to invert on every single clock cycle, the T Flip-Flop is widely used to build binary counters and frequency dividers (it cuts the clock frequency exactly in half).

Circuit Diagram



Fundamental Tables

Truth Table (Function Table):

Clk	T	Q_{n+1}
0	0	Q_n (No Change)
1	1	$\overline{Q}_n$ (Toggle)

Characteristic Table:

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Excitation Table:

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Characteristic Equation

If you look closely at the Characteristic Table, the output Q_{n+1} is `1` only when T and Q_n are *different*. This is the exact definition of an Exclusive-OR (XOR) gate.

$$Q_{n+1} = T \oplus Q_n$$

(Expanded: $Q*{n+1} = T\overline{Q}*n + \overline{T}Q_n$)

---

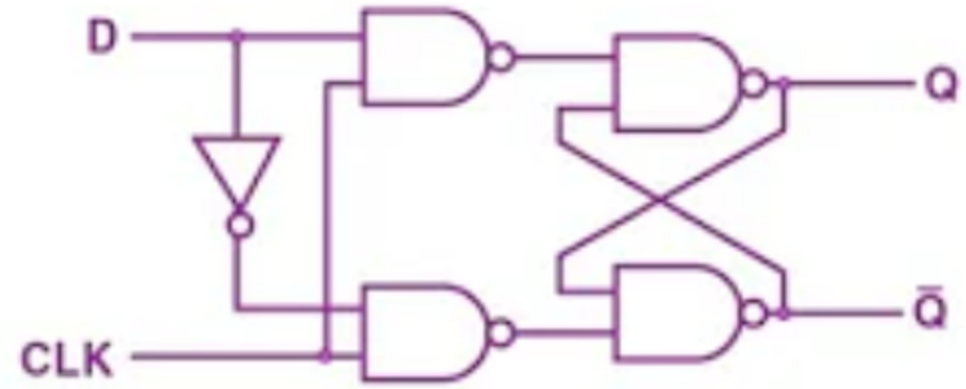
2. The D (Data/Delay) Flip-Flop

The D flip-flop is created by attaching an inverter (NOT gate) between the S and R inputs of an SR Flip-Flop, creating a single input D.

- This guarantees that S and R are always opposites, completely eliminating the Invalid $(1, 1)$ state.
- Whatever data is on the D pin is simply passed through to the Q output on the active clock edge.

Because it just captures and holds the input, it is the standard building block for computer memory (RAM) and data registers.

Circuit Diagram



Fundamental Tables

Truth Table (Function Table):

Clk	D	Q_{n+1}
0	X	Q_n
1	0	0
1	1	1

Characteristic Table:

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

Excitation Table:

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Characteristic Equation

Because the next state (Q_{n+1}) completely ignores the current state (Q_n) and simply copies the D input exactly, the equation is incredibly simple:

$$Q_{n+1} = D$$

Flip-Flop Conversions

The Golden Methodology (4 Steps)

To convert any flip-flop into another type, you are essentially designing a combinational logic circuit that sits in front of the **Available** flip-flop to make it behave like the **Desired** flip-flop.

You must always follow these 4 steps:

1. **Identify:** State what you *have* (Available FF) and what you *want* (Desired FF).
2. **The Conversion Table:** Create a master table. The inputs are the pins of the Desired FF and the current state (Q_n). Use the Desired FF's Characteristic logic to find the Next State (Q_{n+1}). Then, use the Available FF's Excitation Table to figure out what physical inputs are required to make that Q_{n+1} happen.
3. **K-Map Derivation:** Plot the required inputs onto K-Maps to find their simplified Boolean equations.
4. **The Circuit Diagram:** Draw the Available FF and wire its inputs using the logic gates derived in Step 3.

Conversion 1: SR to D Flip-Flop

- **Desired:** D Flip-Flop
- **Available:** SR Flip-Flop

Step 2: The Conversion Table

We use the D Flip-Flop logic ($Q_{n+1} = D$) to find the next state. Then we use the SR Excitation Table to figure out what S and R need to be to cause that transition.

Desired Input	Current State	Next State	Required Available Inputs	
D	Q_n	Q_{n+1}	S	R
0	0	0	0	X
0	1	0	0	1
1	0	1	1	0
1	1	1	X	0

Step 3: K-Map Derivation

From your conversion table, we plot the required inputs S and R onto separate 2-variable Karnaugh maps based on the inputs D and Q_n.

K-Map for S:

$D \backslash Q_n$	0	1
0	0	0
1	1	X

- **Grouping:** We group the `1` and the Don't Care (`X`) in the bottom row ($D = 1$).
- **Equation:** $S = D$

K-Map for R:

$D \backslash Q_n$	0	1
0	X	1
1	0	0

- **Grouping:** We group the `X` and the `1` in the top row ($D = 0$).
- **Equation:** $R = \overline{D}$

If we plot S and R onto separate 2-variable K-Maps (for D and Q_n), we get very simple loops:

- **For S:** A loop of 1s in the $D = 1$ row. Equation:

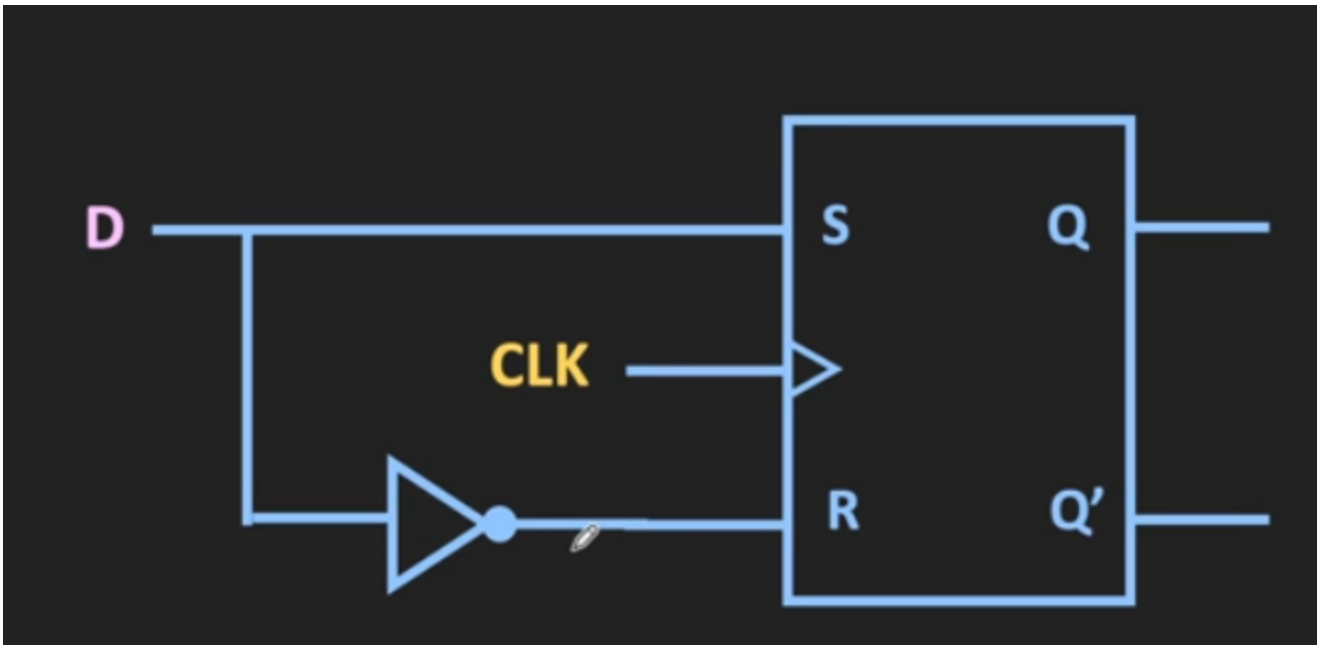
$$S = D$$

- **For R:** A loop of 1s in the $D = 0$ row. Equation:

$$R = \overline{D}$$

Step 4: Circuit Diagram

This mathematically proves the D-Latch/Flip-Flop architecture we covered previously. We simply attach an inverter to the D input and route it to R.



Conversion 2: T to JK Flip-Flop

- **Desired:** JK Flip-Flop
- **Available:** T Flip-Flop

Step 2: The Conversion Table

We use the JK Characteristic logic to determine Q_{n+1}. Then we look at the T Excitation logic (remember, $T = 1$ if the state changes, $T = 0$ if it holds) to find the required T input.

Desired Inputs		Current	Next State	Req. Input
J	K	Q_n	Q_{n+1}	T
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

Step 3: K-Map Derivation

We plot the required T input onto a 3-variable K-Map based on the desired inputs (J, K) and the current state (Q_n).

K-Map for T:

$J\backslash KQ_n$	00	01	11	10
0	0	0	1	0
1	1	0	1	1

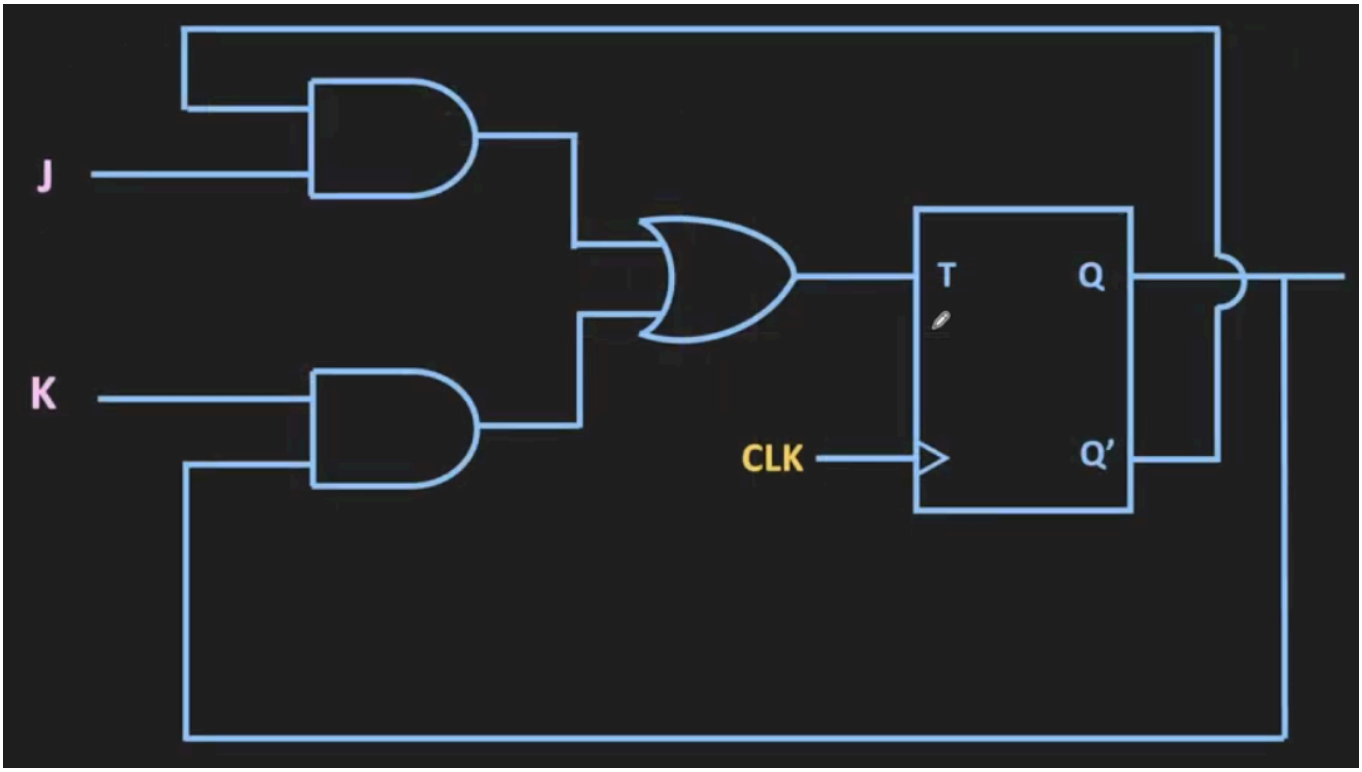
- **Grouping 1:** Group the `1`s at `100` and `110`. This eliminates K. The remaining term is $J\overline{Q}_n$.
- **Grouping 2:** Group the `1`s at `011` and `111`. This eliminates J. The remaining term is KQ_n.
- **Equation:** $T = J\overline{Q}_n + KQ_n$
- Group 1 groups the 1s where $J = 1$ and $Q_n = 0$.
- Group 2 groups the 1s where $K = 1$ and $Q_n = 1$.

Equation:

$$T = J\overline{Q}_n + KQ_n$$

Step 4: Circuit Diagram

We must use two AND gates and an OR gate to feed the correct logic into the T pin.



Conversion 3: SR to JK Flip-Flop

- **Desired:** JK Flip-Flop
- **Available:** SR Flip-Flop

Step 2: The Conversion Table

We map all 8 possible combinations of J, K, Q_n to find Q_{n+1}, and then use the SR Excitation logic to map out the required S and R inputs.

Desired Inputs		Current	Next State	Req. Inputs	
J	K	Q_n	Q_{n+1}	S	R
0	0	0	0	0	X
0	0	1	1	X	0
0	1	0	0	0	X
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	X	0
1	1	0	1	1	0
1	1	1	0	0	1

Step 3: K-Map Derivation

K-Map for S:

$J \backslash KQ_n$	00	01	11	10
0	0	X	0	0
1	1	X	0	1

- **Grouping:** We loop the `1`s at `100` and `110`. We *ignore* the `X`s in the `01` column because looping them wouldn't help us form a valid 4-cell group with the existing `1`s.
- **Equation:** $S = J\overline{Q}_n$

K-Map for R:

$J \backslash KQ_n$	00	01	11	10
0	X	0	1	X
1	0	0	1	0

- **Grouping:** We loop the `1`s at `011` and `111`. We ignore the `X`s in the `00` and `10` columns.
- **Equation:** $R = KQ_n$
- **For S:** We group the 1s and Xs where $J = 1$ and $Q_n = 0$Equation:

$$S = J\overline{Q}_n$$

- **For R:** We group the 1s and Xs where $K = 1$ and $Q_n = 1$Equation:

$$R = KQ_n$$

Step 4: Circuit Diagram

Look closely at this final diagram. By routing the outputs Q and Q' back to the front via AND gates, we have mathematically proven *exactly* how the original JK Flip-Flop was invented to solve the SR Flip-Flop's invalid state!

